

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN



SINH VIÊN THỰC HIỆN

PHẠM TẤN ĐẠT

MSSV: 225633

LỚP: DH22KPM01

TÊN ĐỀ TÀI

**XÂY DỰNG HỆ THỐNG NHẬN DẠNG UNG THƯ DA BẰNG TRÍ
TUỆ NHÂN TẠO**

NGÀNH KỸ THUẬT PHẦN MỀM

Mã ngành: 7480103

Cần Thơ, tháng 3 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN

PHẠM TẤN ĐẠT

MSSV: 225633

LỚP: DH22KPM01

TÊN ĐỀ TÀI

**XÂY DỰNG HỆ THỐNG NHẬN DẠNG UNG THU' DA BẰNG TRÍ
TUỆ NHÂN TẠO**

NGÀNH KỸ THUẬT PHẦN MỀM

Mã ngành: 7480103

GIẢNG VIÊN HƯỚNG DẪN

TRẦN VĂN THIỆN

Cần Thơ, ngày 22 tháng 5 năm 2026

LỜI CẢM TẠ

Chúng em xin chân thành cảm ơn quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Nam Cần Thơ đã tận tình giảng dạy, truyền đạt cho chúng em những kiến thức quý báu trong suốt quá trình học tập và thực hiện đề tài. Nhờ sự hướng dẫn và chỉ bảo tận tâm của quý Thầy Cô, chúng em đã có được nền tảng kiến thức chuyên ngành cần thiết để hoàn thành đồ án này.

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn thầy Trần Văn Thiện, người đã tận tình hướng dẫn, hỗ trợ và giải đáp những khó khăn mà nhóm gặp phải trong suốt quá trình thực hiện đồ án 2. Sự quan tâm, góp ý và định hướng của thầy là nguồn động viên rất lớn, giúp nhóm chúng em hoàn thành bài báo cáo một cách tốt nhất.

Do kiến thức chuyên môn và kinh nghiệm thực tế của chúng em còn hạn chế nên trong quá trình thực hiện đồ án khó tránh khỏi những thiếu sót. Vì vậy, chúng em rất mong nhận được sự thông cảm cùng những ý kiến đóng góp quý báu từ quý Thầy Cô để nhóm có thể rút kinh nghiệm, bổ sung kiến thức và hoàn thiện bản thân hơn trong quá trình học tập sau này.

Cuối lời, chúng em xin kính chúc quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Nam Cần Thơ thật nhiều sức khỏe, hạnh phúc và thành công trong công tác giảng dạy.

em xin chân thành cảm ơn!

Cần Thơ, ngày 22 tháng 5 năm 2026

Sinh viên thực hiện

PHẠM TẤN ĐẠT

LỜI CAM KẾT

tôi xin cam kết đồ án cơ sở kỹ thuật phần mềm này được hoàn thành dựa trên kết quả nghiên cứu của chúng tôi và kết quả này chưa được sử dụng cho bất kỳ đồ án cơ sở nào khác.

Sinh viên thực hiện

PHẠM TẤN ĐẠT

[illegible]

TRẦN VĂN THIÊN

[illegible]

TRẦN VĂN THIÊN

MỤC LỤC

CHƯƠNG 1. MỞ ĐẦU.....	1
1.1. Lý do chọn đề tài.....	1
1.2. Mục tiêu nghiên cứu.....	1
1.3. Đối tượng nghiên cứu.....	2
1.4. Phạm vi nghiên cứu.....	3
1.5. Phương pháp nghiên cứu.....	3
1.6. Ý nghĩa khoa học và thực tiễn của đề tài.....	4
1.6.1. Ý nghĩa khoa học.....	4
1.6.2. Ý nghĩa thực tiễn.....	6
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	8
2.1. Tổng quan về hệ thống hỗ trợ phát hiện tổn thương da từ ảnh số.....	8
2.1.1. Vị trí của hệ thống trong bài toán y tế số.....	8
2.1.2. Mục tiêu tổng quát của hệ thống.....	9
2.1.3. Định hướng tiếp cận của hệ thống.....	9
2.1.4. Cấu trúc tổng quát của hệ thống.....	10
2.1.5. Luồng hoạt động tổng quát.....	11
2.1.6. Đặc điểm riêng của hệ thống trong đề tài.....	11
2.1.7. Vai trò của phân tổng quan này trong toàn bộ chương.....	12
2.1.8. Kết luận chương.....	12
CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ KIẾN TRÚC HỆ THỐNG.....	13
3.1. Mô tả bài toán.....	13
3.2. Phân tích yêu cầu hệ thống.....	14
3.3. Phân tích tác nhân của hệ thống.....	14
3.4. Phân tích use case của hệ thống.....	15
3.5. Kiến trúc tổng thể của hệ thống.....	16
3.6. Thiết kế luồng nghiệp.....	18
3.7. Đánh giá kiến trúc và định hướng phát triển.....	19
3.8. Kết luận chương.....	19
CHƯƠNG 4. THIẾT KẾ HỆ THỐNG.....	20
4.1. Vai trò của backend trong hệ thống.....	20
4.2. Kiến trúc tổng quát của backend.....	21
4.2.1. Lớp giao tiếp API.....	22
4.2.2. Lớp xác thực và kiểm soát truy cập.....	22
4.2.3. Lớp xử lý nghiệp vụ.....	23
4.2.4. Lớp truy cập dữ liệu.....	24
4.2.5. Lớp cấu hình và quan sát hệ thống.....	25
4.3. Thiết kế module xác thực.....	26
4.3.1. Xác thực người dùng bằng JWT.....	26
4.3.2. Xác thực liên kết với Google ID Token.....	26
4.3.3. Phân quyền theo role.....	27
4.3.4. Bảo vệ route và phạm vi công khai.....	27
4.4. Bảng API và endpoint hệ thống.....	27
4.4.1. Controller xác thực.....	28
4.4.2. Controller người dùng.....	29
4.4.3. Controller dự đoán và lịch sử.....	29

4.4.4. Controller phản hồi.....	31
4.4.5. Controller quản trị	31
4.4.6. Nhận xét về cách tổ chức controller và endpoint.....	32
4.4.7. Sơ đồ quan hệ thực thể ERD.....	33
4.5. Cơ sở dữ liệu	33
CHƯƠNG 5. TRAINING AI VÀ TRIỂN KHAI.....	37
5.1. Pipeline mô hình.....	37
5.1.1. Diễn biến hội tụ trên tập validation.....	38
5.1.2. ROC curves	39
5.1.3. Confusion matrix	40
5.2. Thảo luận kết quả	41
5.3. Phân tích mã huấn luyện.....	42
5.4. Môi trường và cấu hình triển khai.....	43
CHƯƠNG 6. ĐÁNH GIÁ KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN.....	44
6.1. Đánh giá kết quả đạt được	44
6.2. Đánh giá so sánh với EfficientNet-B4 và EfficientNet-B5 từ nguồn công khai	45
6.3. Những mặt còn hạn chế	46
6.4. Hướng phát triển trong tương lai.....	47
6.5. Kết luận chương	48
TÀI LIỆU THAM KHẢO	49

DANH MỤC CHỮ VIẾT TẮT

Từ viết tắt	Giải thích
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface – Giao diện lập trình ứng dụng
AMP	Automatic Mixed Precision – Huấn luyện độ chính xác hỗn hợp tự động
CNN	Convolutional Neural Network – Mạng nơ-ron tích chập
DTO	Data Transfer Object – Đối tượng truyền dữ liệu
ERD	Entity Relationship Diagram – Sơ đồ quan hệ thực thể
HAM10000	Human Against Machine with 10000 training images – Bộ dữ liệu ảnh tổn thương da
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
DF	Dermatofibroma – U xơ da
MEL	Melanoma – U hắc tố
NV	Nevus – Nốt ruồi
VASC	Vascular Lesion – Tổn thương mạch máu
BKL	Benign Keratosis – Dày sừng lành tính
BCC	Basal Cell Carcinoma – Ung thư biểu mô tế bào đáy
AKIEC	Actinic Keratosis / Intraepithelial Carcinoma – Dày sừng quang hóa

CHƯƠNG 1:

MỞ ĐẦU

1.1. Lý do chọn đề tài

Trong những năm gần đây, tỷ lệ mắc các bệnh ung thư trên toàn thế giới có xu hướng gia tăng nhanh chóng, trong đó ung thư da là một trong những loại ung thư phổ biến nhất. Theo các nghiên cứu y khoa, việc phát hiện sớm các tổn thương ác tính trên da có vai trò quyết định đến hiệu quả điều trị và khả năng sống sót của bệnh nhân. Tuy nhiên, quá trình chẩn đoán ung thư da hiện nay vẫn phụ thuộc chủ yếu vào kinh nghiệm lâm sàng của bác sĩ chuyên khoa, kết hợp với các phương pháp xét nghiệm cận lâm sàng như sinh thiết hoặc soi da (dermoscopy). Điều này dẫn đến một số hạn chế như tốn thời gian, chi phí cao và khó triển khai rộng rãi tại các khu vực thiếu nguồn nhân lực y tế chuyên môn.

Sự phát triển mạnh mẽ của công nghệ trí tuệ nhân tạo, đặc biệt là lĩnh vực học sâu (Deep Learning), đã mở ra nhiều cơ hội trong việc hỗ trợ chẩn đoán bệnh dựa trên dữ liệu hình ảnh. Các mô hình học sâu hiện đại có khả năng tự động học và trích xuất các đặc trưng phức tạp từ hình ảnh y khoa với độ chính xác cao, từ đó hỗ trợ phân loại bệnh lý một cách hiệu quả. Trong lĩnh vực da liễu, nhiều nghiên cứu đã chứng minh rằng các mô hình mạng nơ-ron tích chập (Convolutional Neural Network – CNN) có thể đạt hiệu năng tương đương hoặc thậm chí vượt qua bác sĩ da liễu trong một số bài toán phân loại tổn thương da.

Bên cạnh đó, sự phổ biến của các thiết bị chụp ảnh có độ phân giải cao và khả năng kết nối Internet đã tạo điều kiện thuận lợi cho việc xây dựng các hệ thống chẩn đoán hỗ trợ từ xa. Người dùng có thể chụp ảnh tổn thương da và gửi lên hệ thống để nhận kết quả phân tích trong thời gian ngắn. Điều này không chỉ giúp tiết kiệm chi phí mà còn góp phần nâng cao khả năng tiếp cận dịch vụ y tế cho người dân ở các khu vực nông thôn hoặc vùng sâu vùng xa.

Xuất phát từ những nhu cầu thực tiễn và tiềm năng ứng dụng của trí tuệ nhân tạo trong lĩnh vực y tế, đề tài “Xây dựng hệ thống nhận diện ung thư da bằng trí tuệ nhân tạo” được lựa chọn nhằm nghiên cứu, phát triển và triển khai một hệ thống hoàn chỉnh có khả năng phân tích hình ảnh tổn thương da và đưa ra dự đoán hỗ trợ chẩn đoán.

1.2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống trí tuệ nhân tạo có khả năng hỗ trợ nhận diện và phân loại các loại tổn thương da dựa trên dữ liệu hình ảnh, từ đó góp phần nâng cao hiệu quả phát hiện sớm ung thư da và hỗ trợ quá trình chẩn đoán lâm sàng. Hệ thống được thiết kế theo hướng tích hợp giữa mô hình học sâu và nền tảng ứng dụng web, cho phép người dùng dễ dàng tải ảnh tổn thương da và nhận kết quả dự đoán trong thời gian ngắn.

Để đạt được mục tiêu tổng quát này, đề tài tập trung vào các mục tiêu cụ thể sau:

- Nghiên cứu cơ sở lý thuyết về học sâu và các kiến trúc mạng nơ-ron tích chập trong bài toán phân loại hình ảnh y khoa, đặc biệt là các mô hình hiện đại thuộc họ EfficientNet.
- Thu thập và xử lý tập dữ liệu hình ảnh tổn thương da HAM10000, bao gồm các bước tiền xử lý, chuẩn hóa dữ liệu và xây dựng pipeline huấn luyện phù hợp với bài toán mất cân bằng dữ liệu.
- Xây dựng và huấn luyện mô hình phân loại tổn thương da dựa trên phương pháp transfer learning, sử dụng kiến trúc EfficientNet-B3 kết hợp các kỹ thuật cải thiện hiệu năng như Weighted Sampling, Mixup augmentation và Focal Loss.
- Đánh giá hiệu năng mô hình thông qua các chỉ số như Macro F1-score, Balanced Accuracy, ROC-AUC và Confusion Matrix nhằm phân tích khả năng phân loại của hệ thống trên từng loại tổn thương.
- Phát triển hệ thống backend sử dụng framework Spring Boot để cung cấp các dịch vụ RESTful API phục vụ việc tải ảnh, xử lý suy luận bằng mô hình AI và lưu trữ kết quả dự đoán.
- Thiết kế giao diện người dùng đơn giản bằng HTML cho phép tương tác với hệ thống, đảm bảo khả năng sử dụng dễ dàng và phản hồi nhanh.
- Xây dựng cơ sở dữ liệu quan hệ nhằm lưu trữ thông tin người dùng, ảnh tổn thương da, kết quả dự đoán và phản hồi đánh giá, từ đó phục vụ cho việc phân tích và cải tiến mô hình trong tương lai.

Thông qua việc hoàn thành các mục tiêu trên, đề tài hướng đến việc xây dựng một hệ thống hỗ trợ chẩn đoán ung thư da có tính ứng dụng thực tiễn cao, đồng thời tạo nền tảng cho các nghiên cứu mở rộng trong lĩnh vực trí tuệ nhân tạo y sinh/

1.3. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là các phương pháp ứng dụng trí tuệ nhân tạo trong bài toán phân loại hình ảnh tổn thương da, cụ thể là việc sử dụng các mô hình học sâu để nhận diện các dạng tổn thương có nguy cơ ung thư. Hệ thống được xây dựng nhằm hỗ trợ phân loại một số loại tổn thương da phổ biến có trong tập dữ liệu HAM10000, bao gồm Melanoma (MEL), Nevus (NV), Basal Cell Carcinoma (BCC), Benign Keratosis (BKL), Dermatofibroma (DF), Vascular Lesion (VASC) và Actinic Keratosis (AKIEC).

Về mặt kỹ thuật, đề tài tập trung nghiên cứu và triển khai mô hình phân loại hình ảnh dựa trên kiến trúc mạng nơ-ron tích chập hiện đại, cụ thể là EfficientNet-B3. Các kỹ thuật huấn luyện nâng cao như transfer learning, xử lý mất cân bằng dữ

liệu và tăng cường dữ liệu (data augmentation) được áp dụng nhằm cải thiện khả năng tổng quát hóa của mô hình. Đề tài không đi sâu vào việc thiết kế kiến trúc mạng mới mà chủ yếu khai thác và tối ưu hóa các mô hình có sẵn cho phù hợp với bài toán thực tế.

1.4. Phạm vi nghiên cứu

Đề tài xây dựng một ứng dụng web cho phép người dùng tải ảnh tổn thương da và nhận kết quả dự đoán từ mô hình trí tuệ nhân tạo. Hệ thống được thiết kế theo kiến trúc client-server với backend xử lý nghiệp vụ và mô hình AI thực hiện suy luận. Việc xây dựng giao diện người dùng chỉ tập trung ở mức chức năng cơ bản nhằm phục vụ quá trình thử nghiệm và đánh giá hệ thống, chưa hướng đến phát triển sản phẩm thương mại hoàn chỉnh.

Về phạm vi triển khai, hệ thống được phát triển và kiểm thử trong môi trường máy chủ thử nghiệm với mục tiêu đảm bảo khả năng suy luận theo thời gian gần thực. Các vấn đề như tối ưu hóa triển khai trên quy mô lớn, tích hợp dữ liệu lâm sàng bổ sung, hoặc chứng nhận y khoa chính thức nằm ngoài phạm vi của đề tài và được đề xuất như các hướng nghiên cứu tiếp theo.

1.5. Phương pháp nghiên cứu

Để đạt được các mục tiêu đề ra, đề tài sử dụng phương pháp nghiên cứu kết hợp giữa nghiên cứu lý thuyết và thực nghiệm triển khai hệ thống. Trước hết, các tài liệu khoa học liên quan đến học sâu, mạng nơ-ron tích chập và các ứng dụng của trí tuệ nhân tạo trong lĩnh vực phân tích hình ảnh y khoa được khảo sát nhằm xây dựng nền tảng lý thuyết cho bài toán. Các công trình nghiên cứu về phân loại tổn thương da sử dụng dataset HAM10000 và các kiến trúc CNN hiện đại như EfficientNet được tham khảo để lựa chọn hướng tiếp cận phù hợp.

Trong giai đoạn thực nghiệm, đề tài áp dụng phương pháp transfer learning nhằm tận dụng các trọng số đã được huấn luyện trước trên tập dữ liệu lớn ImageNet. Cách tiếp cận này giúp rút ngắn thời gian huấn luyện và cải thiện hiệu năng mô hình khi số lượng dữ liệu huấn luyện trong bài toán chuyên biệt còn hạn chế. Quá trình huấn luyện mô hình được thực hiện theo chiến lược hai giai đoạn, bao gồm giai đoạn đóng băng (freeze) các tầng backbone để huấn luyện bộ phân loại cuối và giai đoạn fine-tuning toàn bộ mô hình với tốc độ học thấp hơn nhằm tối ưu hóa đặc trưng cho dữ liệu tổn thương da.

Bên cạnh đó, các kỹ thuật xử lý mất cân bằng dữ liệu được áp dụng nhằm giảm thiên lệch của mô hình đối với các lớp chiếm ưu thế. Phương pháp Weighted Sampling được sử dụng trong quá trình tạo batch huấn luyện để tăng tần suất xuất hiện của các lớp hiếm. Đồng thời, kỹ thuật Mixup augmentation được triển khai nhằm tạo ra các mẫu dữ liệu trung gian giúp cải thiện khả năng tổng quát hóa. Hàm

mất mát Focal Loss kết hợp với label smoothing được sử dụng nhằm tăng độ nhạy đối với các mẫu khó phân loại.

Việc đánh giá hiệu năng mô hình được thực hiện thông qua các chỉ số phù hợp với bài toán phân loại đa lớp mất cân bằng như Macro F1-score, Balanced Accuracy và diện tích dưới đường cong ROC (ROC-AUC). Ngoài ra, ma trận nhầm lẫn (Confusion Matrix) được sử dụng để phân tích chi tiết các trường hợp dự đoán sai giữa các loại tổn thương có đặc trưng hình thái tương tự.

Song song với việc xây dựng mô hình học sâu, đề tài còn áp dụng phương pháp phát triển phần mềm theo hướng mô hình hóa hệ thống và thiết kế kiến trúc client-server. Backend được triển khai bằng framework Spring Boot nhằm cung cấp các dịch vụ RESTful API phục vụ việc tiếp nhận ảnh đầu vào, xử lý suy luận và trả kết quả dự đoán. Mô hình trí tuệ nhân tạo được tích hợp dưới dạng dịch vụ suy luận riêng biệt nhằm đảm bảo tính linh hoạt và khả năng mở rộng của hệ thống trong quá trình triển khai.

1.6. Ý nghĩa khoa học và thực tiễn của đề tài

Đề tài xây dựng hệ thống hỗ trợ phân loại tổn thương da từ ảnh số mang ý nghĩa cả về mặt khoa học lẫn thực tiễn, bởi nó không chỉ dừng ở việc huấn luyện một mô hình nhận dạng ảnh, mà còn tổ chức một quy trình tương đối hoàn chỉnh gồm tiếp nhận dữ liệu ảnh, suy luận mô hình, lưu trữ lịch sử, thu nhận phản hồi người dùng và hỗ trợ quản trị dữ liệu phục vụ cải tiến hệ thống. Trong phạm vi dự án này, đối tượng phân loại là bộ dữ liệu HAM10000 với 7 nhóm tổn thương da, đồng thời mô hình được thiết kế theo định hướng xử lý tốt bài toán mất cân bằng lớp và giảm hiện tượng quá khớp trên dữ liệu y tế có quy mô không quá lớn như mô tả trong README của dự án.

1.6.1. Ý nghĩa khoa học

Về phương diện khoa học, đề tài có ý nghĩa trước hết ở chỗ nó tiếp cận một bài toán thị giác máy tính trong miền y sinh, nơi mà dữ liệu không chỉ phức tạp về mặt hình ảnh mà còn nhạy cảm về mặt suy luận. Ảnh tổn thương da thường có sự biến thiên lớn về màu sắc, hình thái, ranh giới, điều kiện chiếu sáng, thiết bị chụp và sắc tố da; trong khi đó, sự khác biệt giữa các lớp bệnh lý đôi khi rất nhỏ. Điều này làm cho bài toán phân loại ảnh da liễu trở thành một tình huống điển hình để kiểm nghiệm giá trị của học sâu trong môi trường dữ liệu khó, không cân bằng và có rủi ro cao nếu dự đoán sai.

Một đóng góp có ý nghĩa của đề tài là lựa chọn hướng tiếp cận dựa trên học chuyển giao. Thay vì xây dựng mô hình từ đầu, hệ thống tận dụng tri thức đã học từ dữ liệu ảnh quy mô lớn, sau đó tinh chỉnh cho miền ảnh da liễu. Đây là một lựa chọn có cơ sở lý thuyết rõ ràng: trong các bài toán y sinh, số lượng dữ liệu gán nhãn thường hạn chế, do đó học chuyển giao giúp giảm nhu cầu dữ liệu huấn luyện quá

lớn, rút ngắn thời gian hội tụ và cải thiện khả năng khái quát hóa. Dự án áp dụng backbone EfficientNet-B3, huấn luyện theo hai giai đoạn: giai đoạn đầu ổn định bộ phân loại đầu ra, giai đoạn sau tinh chỉnh toàn bộ mạng với tốc độ học nhỏ hơn, như mô tả trong README và phần triển khai suy luận trong ModelService. Cách làm này phù hợp với tinh thần của nghiên cứu EfficientNet, vốn nhấn mạnh việc mở rộng mô hình theo cách cân bằng giữa độ sâu, độ rộng và độ phân giải để đạt hiệu quả tốt hơn trên bài toán phân loại ảnh (Tan & Le, 2019).

Điều có giá trị khoa học tiếp theo là đề tài không xem độ chính xác tổng thể là thước đo duy nhất, mà chú ý đến bản chất mất cân bằng lớp trong dữ liệu da liễu. Với HAM10000, một số nhóm tổn thương phổ biến xuất hiện nhiều hơn rất rõ rệt so với các nhóm hiếm, vì vậy nếu chỉ tối ưu theo độ chính xác chung thì mô hình dễ nghiêng về các lớp nhiều mẫu. Tài liệu dự án cho thấy hệ thống đã kết hợp các giải pháp như lấy mẫu cân bằng, focal loss, tăng cường dữ liệu, mixup, label smoothing, dừng sớm và các kỹ thuật ổn định tối ưu hóa để cải thiện khả năng học trên nhóm hiếm và giảm quá khớp README. Về lý thuyết, focal loss được đề xuất để làm giảm ảnh hưởng của các mẫu “dễ”, từ đó buộc mô hình tập trung hơn vào các mẫu khó hoặc lớp ít gặp (Lin et al., 2017). Mixup lại góp phần làm mượt biên quyết định và giảm hiện tượng ghi nhớ cứng dữ liệu huấn luyện (Zhang et al., 2018). AdamW có ý nghĩa trong việc tách weight decay khỏi cơ chế cập nhật tham số, giúp quá trình regularization hiệu quả và ổn định hơn trên mạng sâu (Loshchilov & Hutter, 2019). Vì vậy, đề tài không chỉ dùng mô hình học sâu như một công cụ, mà còn kiểm nghiệm một tổ hợp kỹ thuật có cơ sở lý thuyết rõ ràng cho bài toán phân loại y sinh.

Một ý nghĩa khoa học quan trọng khác nằm ở cách đánh giá mô hình. Theo các biểu đồ kết quả mà nhóm cung cấp, hệ thống không chỉ theo dõi loss trong huấn luyện và kiểm định, mà còn quan tâm tới macro-F1, balanced accuracy, ROC-AUC theo từng lớp và ma trận nhầm lẫn. Cách đánh giá này phù hợp với đặc trưng dữ liệu y học, nơi mà chỉ số trung bình đơn thuần có thể che khuất sai sót ở những lớp ít mẫu nhưng có ý nghĩa lâm sàng cao. Macro-F1 phản ánh công bằng hơn giữa các lớp; balanced accuracy giúp giảm thiên lệch do phân bố dữ liệu; ROC-AUC theo từng lớp cho thấy khả năng phân biệt của mô hình ở từng nhóm bệnh; còn ma trận nhầm lẫn cho phép quan sát trực tiếp các cặp tổn thương dễ bị nhầm lẫn. Xét trên logic nghiên cứu, đây là cách đánh giá chặt chẽ hơn so với việc chỉ báo cáo một con số accuracy duy nhất.

Về phương diện hệ thống, đề tài còn có ý nghĩa khoa học ở chỗ nó nối kết giữa mô hình trí tuệ nhân tạo và quy trình quản lý dữ liệu suy luận. Kết quả dự đoán không bị “mất” sau mỗi lần chạy mà được lưu trữ cùng thông tin ảnh tải lên, xác suất dự đoán, kết quả top-k và thời điểm thực hiện; đồng thời hệ thống còn thu nhận phản hồi từ người dùng và cho phép quản trị viên chọn mẫu phục vụ tái huấn luyện

qua các API và giao diện quản trị đã triển khai trong backend và frontend của dự án PredictionService, FeedbackController, AdminController, index.html. Điều này tạo nên một vòng lặp học tập có phản hồi, rất gần với hướng phát triển của các hệ thống AI ứng dụng thực tế: mô hình không chỉ dự đoán một lần, mà còn có khả năng được cải thiện dần nhờ dữ liệu phát sinh trong quá trình vận hành. Sau cùng, đề tài có giá trị khoa học vì nó minh họa khá rõ cho xu hướng AI có thể triển khai được, tức là không dừng ở mức thí nghiệm thuật toán mà còn tổ chức thành kiến trúc nhiều thành phần: lớp giao diện người dùng, lớp dịch vụ nghiệp vụ và lớp dịch vụ suy luận mô hình. Trong dự án, phần nhận diện được tách thành một dịch vụ độc lập, còn phần quản lý người dùng, xác thực, lịch sử và phản hồi được triển khai ở lớp backend riêng; cấu trúc này được mô tả trong README, cấu hình triển khai docker-compose, Dockerfile backend và Dockerfile ModelService. Xét về học thuật, đây là một minh chứng cho việc nghiên cứu AI hiện đại cần gắn với khả năng tái lập, triển khai và mở rộng, thay vì chỉ trình bày mô hình trên lý thuyết.

1.6.2. Ý nghĩa thực tiễn

Về mặt thực tiễn, đề tài có giá trị trước hết ở khả năng xây dựng một công cụ hỗ trợ sàng lọc ban đầu đối với các tổn thương da từ ảnh chụp. Trong điều kiện người dùng phổ thông thường khó tự phân biệt các biểu hiện ngoài da, hệ thống có thể cung cấp một nhận định tham khảo nhanh dựa trên ảnh tải lên, từ đó giúp người dùng chú ý sớm hơn tới các dấu hiệu bất thường và cân nhắc đi khám chuyên khoa khi cần. Cần nhấn mạnh rằng chức năng này mang tính hỗ trợ, không thay thế kết luận của bác sĩ; tuy nhiên, ở góc độ ứng dụng, đây là bước rất hữu ích để nâng cao nhận thức, giảm tâm lý chủ quan và cải thiện khả năng tiếp cận thông tin sức khỏe ban đầu.

Một giá trị thực tiễn nổi bật của đề tài là nó không chỉ cho ra một nhận dự đoán, mà còn tổ chức quy trình sử dụng tương đối hoàn chỉnh cho người dùng cuối. Hệ thống có cơ chế đăng nhập, cập nhật hồ sơ, tải ảnh, xem lịch sử dự đoán, phản hồi mức độ đúng sai và theo dõi kết quả gần nhất ngay trên giao diện web AuthController, UserController, PredictionController, index.html. Điều này cho thấy đề tài hướng đến trải nghiệm sử dụng thật, chứ không chỉ là một đoạn mã kiểm thử mô hình trong môi trường nghiên cứu. Trong bối cảnh nhiều đề án AI chỉ dừng ở notebook hoặc giao diện minh họa đơn giản, việc phát triển một luồng sử dụng đầy đủ làm tăng rõ rệt giá trị ứng dụng của hệ thống.

Đề tài còn có ý nghĩa thực tiễn ở khả năng lưu vết và quản trị dữ liệu. Khi người dùng tải ảnh lên, hệ thống không chỉ gửi ảnh đi dự đoán mà còn lưu thông tin ảnh, dung lượng, giá trị băm, thời gian tải lên và kết quả dự đoán tương ứng vào cơ sở dữ liệu ImageUpload, Prediction, PredictionService. Về thực tiễn, điều này rất quan trọng vì một hệ thống chăm sóc sức khỏe số cần có khả năng truy xuất kết quả,

theo dõi lịch sử tương tác và tổ chức dữ liệu có hệ thống. Nó giúp việc kiểm tra lại, so sánh lần dự đoán trước và sau, cũng như thu thập dữ liệu phục vụ cải tiến thuật toán trở nên khả thi.

Một khía cạnh thực tiễn khác là hệ thống đã dự liệu vai trò quản trị thay vì chỉ phục vụ người dùng cá nhân. Từ giao diện và các API quản trị, quản trị viên có thể xem tổng hợp số liệu, danh sách người dùng, kết quả dự đoán của từng người, ảnh đã tải lên, phản hồi và trạng thái dừng lại dữ liệu cho huấn luyện AdminController, index.html. Điều này có ý nghĩa lớn nếu sau này hệ thống được phát triển thành công cụ hỗ trợ tại phòng khám, cơ sở nghiên cứu hoặc môi trường đào tạo, nơi cần theo dõi chất lượng dự đoán trên tập người dùng thực và kiểm soát quá trình hình thành dữ liệu phản hồi.

Từ góc nhìn triển khai, đề tài có tính thực tiễn khá cao vì được tổ chức theo cấu trúc dịch vụ tách biệt, có cấu hình đóng gói container và khả năng kết nối cơ sở dữ liệu độc lập docker-compose, application.yml. Điều này mang lại ba lợi ích thực tế. Thứ nhất, hệ thống dễ triển khai trên máy chủ hoặc nền tảng đám mây. Thứ hai, các thành phần có thể bảo trì độc lập, ví dụ cập nhật mô hình mà không phải thay đổi toàn bộ giao diện và nghiệp vụ. Thứ ba, cấu trúc này thuận lợi cho mở rộng sau này, chẳng hạn thay mô hình mới, thêm cơ chế kiểm tra chất lượng ảnh, hoặc bổ sung chức năng tư vấn, thống kê và cảnh báo. Với một đồ án ứng dụng AI, khả năng triển khai được là tiêu chí thực tế rất quan trọng.

Đề tài cũng có ý nghĩa thực tiễn đối với đào tạo và nghiên cứu liên ngành. Nó là ví dụ rõ ràng về sự kết hợp giữa trí tuệ nhân tạo, công nghệ phần mềm, quản trị cơ sở dữ liệu và xây dựng dịch vụ web. Thông qua dự án này, người học có thể tiếp cận trọn vẹn một chu trình phát triển hệ thống AI ứng dụng: từ tiền xử lý dữ liệu, lựa chọn mô hình, đánh giá thuật toán, xây dựng API, thiết kế giao diện, tích hợp xác thực người dùng cho tới triển khai hệ thống. Vì vậy, ngoài giá trị sử dụng trực tiếp, đề tài còn có giá trị như một mô hình tham khảo cho các nhóm nghiên cứu hoặc sinh viên trong những bài toán tương tự thuộc lĩnh vực y tế số.

Cuối cùng, xét trong bối cảnh chuyển đổi số y tế, đề tài mang ý nghĩa thực tiễn ở chỗ góp phần định hình cách tiếp cận hỗ trợ ra quyết định bằng AI, chứ không thay thế chuyên gia. Logic mà hệ thống lựa chọn là hợp lý: người dùng cung cấp ảnh, hệ thống trả về kết quả phân loại kèm độ tin cậy và nhiều phương án top-k từ dịch vụ suy luận ModelService main, ModelService, sau đó lưu lịch sử và thu phản hồi để cải thiện dần chất lượng. Cách làm này phù hợp với yêu cầu thực tế của các hệ thống AI trong y tế: phải minh bạch hơn, có khả năng theo dõi, có phản hồi và có không gian để con người can thiệp.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Tổng quan về hệ thống hỗ trợ phát hiện tổn thương da từ ảnh số

Trong bối cảnh chuyển đổi số y tế ngày càng phát triển, các hệ thống hỗ trợ chẩn đoán dựa trên trí tuệ nhân tạo đang trở thành một hướng nghiên cứu và ứng dụng có giá trị cao. Đối với lĩnh vực da liễu, nhu cầu xây dựng các công cụ hỗ trợ nhận diện tổn thương da từ ảnh số xuất phát từ thực tế rằng nhiều biểu hiện ngoài da có hình thái khá giống nhau, khó phân biệt bằng cảm quan thông thường, trong khi việc phát hiện sớm lại có ý nghĩa rất lớn đối với hiệu quả theo dõi và điều trị. Trên cơ sở đó, đề tài xây dựng một hệ thống hỗ trợ phân loại tổn thương da từ ảnh, sử dụng bộ dữ liệu gồm 7 nhóm tổn thương phổ biến, đồng thời triển khai thành một mô hình ứng dụng có khả năng tiếp nhận ảnh, phân tích, trả kết quả, lưu lịch sử và tiếp nhận phản hồi người dùng.

Về bản chất, đây là một hệ thống hỗ trợ quyết định, không phải hệ thống thay thế bác sĩ. Vai trò của nó là tiếp nhận ảnh tổn thương da, thực hiện suy luận bằng mô hình học sâu, sau đó đưa ra dự đoán kèm theo mức độ tin cậy để người dùng hoặc người quản lý có thêm một nguồn tham khảo ban đầu. Ý nghĩa của cách tiếp cận này nằm ở chỗ hệ thống không cố gắng đưa ra kết luận y khoa cuối cùng, mà đóng vai trò như một tầng hỗ trợ sàng lọc, góp phần giảm tải bước đánh giá sơ bộ, nâng cao nhận thức ban đầu và hỗ trợ tổ chức dữ liệu phục vụ theo dõi lâu dài.

2.1.1. Vị trí của hệ thống trong bài toán y tế số

Trong các hệ thống sức khỏe số hiện nay, dữ liệu hình ảnh là một dạng dữ liệu có giá trị nhưng cũng rất khó xử lý. Đối với ảnh da liễu, cùng một loại tổn thương có thể thay đổi đáng kể theo ánh sáng, góc chụp, sắc tố da, kích thước vùng bệnh, chất lượng thiết bị ghi ảnh và điều kiện môi trường. Ngược lại, các lớp bệnh lý khác nhau đôi khi lại có đặc điểm hình ảnh gần nhau, khiến việc phân biệt trở nên phức tạp. Vì vậy, bài toán nhận diện tổn thương da từ ảnh không chỉ đơn thuần là nhận dạng mẫu hình, mà còn là bài toán phân loại trong điều kiện biến thiên lớn của dữ liệu đầu vào.

Từ góc nhìn đó, hệ thống của đề tài được đặt ở giao điểm giữa ba lĩnh vực chính. Thứ nhất là thị giác máy tính, nơi ảnh đầu vào được chuyển thành đặc trưng để mô hình học cách phân biệt các nhóm tổn thương. Thứ hai là trí tuệ nhân tạo ứng dụng trong y tế, nơi kết quả suy luận phải được tổ chức theo hướng hỗ trợ, có khả năng giải thích tương đối và hạn chế rủi ro sử dụng sai. Thứ ba là công nghệ phần mềm, nơi mô hình không tồn tại riêng lẻ trong môi trường thử nghiệm mà được tích hợp vào một hệ thống hoàn chỉnh có người dùng, dữ liệu, lịch sử thao tác và quy trình quản trị. Đây cũng chính là điểm nổi bật của đề tài: không dừng ở việc huấn

luyện mô hình, mà phát triển thành một hệ thống có tính ứng dụng tương đối đầy đủ, gồm giao diện, dịch vụ xử lý nghiệp vụ, dịch vụ suy luận và cơ sở dữ liệu lưu trữ kết quả .

2.1.2. Mục tiêu tổng quát của hệ thống

Mục tiêu tổng quát của hệ thống là xây dựng một nền tảng có khả năng hỗ trợ phát hiện và phân loại tổn thương da từ ảnh số theo quy trình khép kín, bao gồm các bước: tiếp nhận dữ liệu ảnh, thực hiện suy luận bằng mô hình trí tuệ nhân tạo, phản hồi kết quả cho người dùng, lưu trữ lịch sử dự đoán, và thu nhận phản hồi để phục vụ cải tiến sau này. Theo mô tả của dự án, hệ thống được thiết kế để xử lý bài toán phân loại 7 lớp tổn thương da, đồng thời chú trọng tới hai vấn đề quan trọng của dữ liệu y sinh là mất cân bằng lớp và nguy cơ quá khớp trên tập dữ liệu không quá lớn .

Nếu xét ở mức chức năng, hệ thống hướng tới bốn mục tiêu chính. Mục tiêu thứ nhất là hỗ trợ nhận diện nhanh. Người dùng có thể cung cấp ảnh tổn thương da và nhận lại kết quả dự đoán trong thời gian ngắn. Điều này tạo ra một bước sàng lọc sơ bộ có ích trong môi trường số.

Mục tiêu thứ hai là quản lý được lịch sử và dữ liệu suy luận. Kết quả không chỉ hiển thị tức thời mà còn được lưu lại để tra cứu sau này. Đây là điểm rất quan trọng nếu muốn hệ thống có giá trị sử dụng thực tế thay vì chỉ là một chương trình minh họa.

Mục tiêu thứ ba là hình thành cơ chế phản hồi. Khi người dùng hoặc quản trị viên đánh giá kết quả là đúng hay sai, dữ liệu phản hồi đó có thể trở thành cơ sở để chọn lọc mẫu phục vụ giai đoạn cải tiến mô hình sau này. Điều này giúp hệ thống có tiềm năng phát triển theo hướng học hỏi dần từ quá trình vận hành thực tế.

Mục tiêu thứ tư là **sẵn sàng triển khai**. Hệ thống được tổ chức theo mô hình tách thành nhiều thành phần tương đối độc lập, giúp thuận lợi hơn trong đóng gói, triển khai, mở rộng và bảo trì .

2.1.3. Định hướng tiếp cận của hệ thống

Khác với cách xây dựng một mô hình học máy đơn lẻ rồi dừng ở bước đánh giá chỉ số, hệ thống trong đề tài được xây dựng theo định hướng AI gắn với ứng dụng thực tế. Theo đó, trí tuệ nhân tạo chỉ là một lõi chức năng trong toàn bộ hệ thống. Bao quanh lõi này là các lớp chức năng phục vụ người dùng và quản trị, như xác thực tài khoản, cập nhật hồ sơ, tải ảnh, xem lịch sử dự đoán, gửi phản hồi và theo dõi dữ liệu trên giao diện quản trị .

Định hướng tiếp cận này phản ánh một logic khá rõ ràng.

Thứ nhất, một hệ thống AI trong y tế không thể chỉ tạo ra kết quả dự đoán, mà còn phải quản lý được toàn bộ vòng đời của kết quả đó. Nghĩa là cần biết ảnh nào đã được gửi lên, kết quả gì đã trả về, ai là người gửi, khi nào gửi, phản hồi sau đó ra sao, và dữ liệu nào có thể được dùng để nâng cao mô hình.

Thứ hai, hệ thống cần phân tách rõ giữa nghiệp vụ ứng dụng và suy luận mô hình. Việc tách phần mô hình thành một dịch vụ riêng giúp giảm phụ thuộc giữa khối AI và khối ứng dụng web. Khi đó, giao diện người dùng và phần quản lý nghiệp vụ có thể được duy trì ổn định, trong khi mô hình nhận diện có thể được thay đổi hoặc nâng cấp mà không làm đứt gãy toàn hệ thống. Kiến trúc của dự án thể hiện rõ điều này: phần giao diện được phục vụ dưới dạng web client, phần xử lý nghiệp vụ được triển khai bằng backend riêng, còn phần suy luận mô hình được tổ chức như một dịch vụ độc lập chuyên nhận ảnh và trả về kết quả phân loại.

Thứ ba, hệ thống được xây dựng theo hướng ưu tiên khả năng sử dụng và khả năng mở rộng, chứ không chỉ tối ưu thuật toán ở mức hẹp. Điều này thể hiện ở việc có cơ chế người dùng, lịch sử dự đoán, phản hồi, vai trò quản trị và triển khai bằng container. Về mặt lý luận hệ thống, đây là hướng tiếp cận phù hợp với một bài toán AI ứng dụng, vì mô hình chỉ thực sự có giá trị khi nó được đưa vào một quy trình vận hành có tổ chức.

2.1.4. Cấu trúc tổng quát của hệ thống

Về cấu trúc, hệ thống có thể được xem như gồm bốn lớp chức năng chính. Lớp thứ nhất là lớp tương tác người dùng. Đây là nơi người dùng thao tác với hệ thống: đăng nhập, khai báo thông tin cơ bản, tải ảnh cần kiểm tra, nhận kết quả dự đoán và xem lại lịch sử. Ngoài ra, hệ thống còn có khu vực dành cho người quản trị nhằm theo dõi số liệu tổng hợp, danh sách người dùng, lịch sử dự đoán theo từng tài khoản và trạng thái phản hồi dữ liệu.

Lớp thứ hai là lớp xử lý nghiệp vụ. Lớp này chịu trách nhiệm điều phối các chức năng của hệ thống như xác thực tài khoản, quản lý hồ sơ, tiếp nhận yêu cầu tải ảnh, lưu thông tin ảnh, gọi dịch vụ suy luận, ghi nhận kết quả và trả dữ liệu về cho giao diện. Các chức năng nghiệp vụ cũng là nơi bảo đảm rằng quá trình tương tác với mô hình được đặt trong một ngữ cảnh ứng dụng hoàn chỉnh, có kiểm soát và có khả năng truy vết.

Lớp thứ ba là lớp suy luận trí tuệ nhân tạo. Đây là nơi mô hình học sâu thực hiện việc phân tích ảnh và xuất ra kết quả dự đoán. Dịch vụ này không đảm nhận vai trò quản trị người dùng hay lưu trữ lịch sử, mà tập trung vào một nhiệm vụ hẹp hơn nhưng cốt lõi hơn: chuyển đổi ảnh đầu vào thành xác suất và nhãn phân loại. Việc tách riêng dịch vụ suy luận như vậy tạo điều kiện thuận lợi cho việc tối ưu hiệu năng, thay mô hình hoặc mở rộng hệ thống sau này.

Lớp thứ tư là lớp lưu trữ và quản trị dữ liệu. Ở đây hệ thống ghi nhận thông tin người dùng, ảnh tải lên, lịch sử dự đoán và phản hồi. Cơ sở dữ liệu không chỉ đóng vai trò lưu trữ, mà còn là nền tảng để hệ thống hình thành trí nhớ vận hành, hỗ trợ thống kê, quản lý người dùng, theo dõi chất lượng mô hình và khai thác các mẫu phục vụ tái huấn luyện về sau.

Nhìn chung, đây là một kiến trúc hợp lý cho bài toán của đề tài, bởi nó cho phép phân chia trách nhiệm rõ ràng: giao diện lo tương tác, backend lo nghiệp vụ, AI service lo suy luận, còn cơ sở dữ liệu lo lưu trữ và truy xuất. Cách tổ chức này giúp hệ thống dễ kiểm soát hơn về mặt logic và thuận lợi hơn khi triển khai thực tế.

2.1.5. Luồng hoạt động tổng quát

Về logic hoạt động, hệ thống có thể được mô tả theo một chuỗi bước tương đối mạch lạc.

Đầu tiên, người dùng truy cập giao diện hệ thống, thực hiện đăng nhập và cung cấp một số thông tin hồ sơ cơ bản. Sau đó, người dùng tải ảnh tổn thương da lên hệ thống và gửi yêu cầu kiểm tra. Ở bước tiếp theo, ảnh được tiếp nhận, lưu trữ và đồng thời được chuyển sang khối suy luận trí tuệ nhân tạo để tiến hành phân loại. Kết quả trả về bao gồm nhãn dự đoán chính và có thể kèm theo nhiều khả năng xếp hạng cao nhất. Sau khi có kết quả, hệ thống vừa hiển thị cho người dùng, vừa lưu vào cơ sở dữ liệu để hình thành lịch sử tra cứu. Trong các bước tiếp theo, người dùng có thể xem lại các lần dự đoán trước đó hoặc gửi phản hồi về độ chính xác của kết quả. Ở phía quản trị, hệ thống cung cấp các chức năng tổng hợp, thống kê và lọc dữ liệu phản hồi để tạo nguồn dữ liệu phục vụ cải thiện mô hình trong tương lai.

Điểm đáng chú ý trong luồng xử lý này là hệ thống không xem dự đoán là điểm kết thúc, mà xem đó là một mắt xích trong chu trình dài hơn gồm: dự đoán – lưu trữ – phản hồi – cải tiến. Đây là một tư duy hệ thống phù hợp với AI ứng dụng thực tế, đặc biệt trong môi trường dữ liệu biến động như y tế.

2.1.6. Đặc điểm riêng của hệ thống trong đề tài

So với nhiều bài toán phân loại ảnh thông thường, hệ thống của đề tài có một số đặc điểm riêng đáng lưu ý.

Trước hết, hệ thống được phát triển trên một miền dữ liệu nhạy cảm và khó, đó là ảnh tổn thương da. Điều này làm tăng yêu cầu về tính ổn định, khả năng khái quát và sự cân trọng trong diễn giải kết quả.

Thứ hai, hệ thống không chỉ phục vụ một thao tác nhận diện đơn lẻ mà còn tích hợp chức năng người dùng, quản trị và phản hồi. Nghĩa là nó đã vượt ra khỏi phạm vi của một mô hình thử nghiệm thuần túy để tiến gần hơn tới một phần mềm ứng dụng.

Thứ ba, hệ thống được định hướng xử lý tốt hơn đối với hiện tượng mất cân bằng lớp và rủi ro quá khớp, vốn là hai vấn đề rất phổ biến trong dữ liệu y tế. Theo mô tả dự án, mô hình được xây dựng trên nền tảng học chuyển giao, huấn luyện nhiều giai đoạn và kết hợp các kỹ thuật tăng cường khả năng tổng quát hóa. Điều này cho thấy mục tiêu của đề tài không chỉ là “làm cho chạy được”, mà còn là “làm cho hợp lý về mặt học thuật và ổn định hơn về mặt thực nghiệm”.

Thứ tư, hệ thống có khả năng triển khai tương đối rõ ràng nhờ cách tổ chức thành các thành phần độc lập và có cấu hình đóng gói riêng. Đây là nền tảng quan trọng nếu muốn đưa hệ thống từ môi trường thử nghiệm sang môi trường trình diễn hoặc vận hành thực tế ở quy mô nhỏ.

2.1.7. Vai trò của phần tổng quan này trong toàn bộ chương

Trong kết cấu của Chương 2, mục tổng quan có vai trò đặt nền cho các mục lý thuyết và công nghệ phía sau. Nếu các mục tiếp theo sẽ đi sâu vào mô hình học sâu, xử lý ảnh, kỹ thuật huấn luyện, cơ sở dữ liệu, dịch vụ web và công nghệ triển khai, thì mục này có nhiệm vụ trả lời câu hỏi ở mức khái quát hơn: hệ thống được xây dựng để giải quyết bài toán gì, được tổ chức theo logic nào, vì sao phải tách thành nhiều lớp chức năng và vì sao bài toán này cần tiếp cận như một hệ thống hoàn chỉnh thay vì chỉ là một mô hình nhận dạng ảnh đơn lẻ.

Nói cách khác, đây là phần chuyển tiếp từ bối cảnh bài toán sang nền tảng lý thuyết và công nghệ. Nhờ có cái nhìn tổng quan này, các nội dung sau sẽ không bị rời rạc, mà được đặt vào đúng vị trí của chúng trong một chỉnh thể thống nhất.

2.1.8. Kết luận chương

Tóm lại, hệ thống của đề tài là một hệ thống hỗ trợ phát hiện tổn thương da từ ảnh số theo định hướng ứng dụng thực tế. Hệ thống không chỉ sử dụng mô hình trí tuệ nhân tạo để phân loại ảnh, mà còn tổ chức đầy đủ các thành phần phục vụ người dùng, lưu trữ dữ liệu, quản lý lịch sử, tiếp nhận phản hồi và hỗ trợ quản trị. Về mặt logic, đây là một kiến trúc phù hợp cho bài toán AI trong y tế, vì nó dung hòa được ba yêu cầu: có khả năng nhận diện tự động, có khả năng quản lý dữ liệu vận hành và có khả năng mở rộng trong tương lai. Chính từ nền tảng tổng quan này, các cơ sở lý thuyết về học sâu, xử lý ảnh, kiến trúc web, cơ sở dữ liệu và công nghệ triển khai mới trở nên có ý nghĩa trong toàn bộ đề tài.

CHƯƠNG 3:

PHÂN TÍCH YÊU CẦU VÀ KIẾN TRÚC HỆ THỐNG

3.1. Mô tả bài toán

Đề tài hướng tới xây dựng một hệ thống hỗ trợ phát hiện và phân loại tổn thương da từ ảnh số, trong đó người dùng có thể tải ảnh lên giao diện web, hệ thống tiếp nhận ảnh, chuyển đến khối trí tuệ nhân tạo để phân tích, sau đó trả về kết quả dự đoán và lưu lại lịch sử sử dụng. Về bản chất, đây không chỉ là bài toán huấn luyện một mô hình nhận dạng ảnh, mà là bài toán xây dựng một hệ thống ứng dụng hoàn chỉnh có khả năng phục vụ người dùng, lưu trữ dữ liệu, quản trị thông tin và hỗ trợ cải tiến mô hình trong tương lai. Theo mô tả của dự án, hệ thống làm việc với bộ dữ liệu HAM10000 gồm 7 nhóm tổn thương da và được thiết kế theo hướng tăng khả năng khái quát hóa, giảm ảnh hưởng của mất cân bằng lớp và hạn chế quá khớp trên dữ liệu y tế có quy mô không quá lớn.

Bài toán đặt ra có nhiều đặc thù khác với các hệ thống nhận dạng ảnh thông thường. Ảnh da liễu có độ biến thiên lớn về màu sắc, ánh sáng, chất lượng ảnh, góc chụp, kích thước vùng tổn thương và nền da của từng người. Trong khi đó, sự khác biệt hình thái giữa các nhóm tổn thương đôi khi không rõ ràng, làm tăng nguy cơ nhầm lẫn. Do đó, hệ thống không thể chỉ chú trọng vào việc “dự đoán được nhãn”, mà còn cần được tổ chức theo hướng đủ ổn định, có khả năng lưu vết, phản hồi và quản trị dữ liệu để hỗ trợ việc đánh giá chất lượng mô hình sau khi triển khai.

Từ góc nhìn ứng dụng, hệ thống phải giải quyết đồng thời nhiều nhu cầu. Người dùng cần một công cụ có thể sử dụng thuận tiện qua web, tải ảnh nhanh, nhận kết quả trong thời gian ngắn và xem lại các lần kiểm tra trước đó. Ở phía quản trị, hệ thống cần cung cấp cái nhìn tổng quát về người dùng, kết quả dự đoán, phản hồi và khả năng chọn mẫu phục vụ tái huấn luyện. Từ góc nhìn kỹ thuật, hệ thống phải tách biệt giữa phần giao diện, phần xử lý nghiệp vụ, phần suy luận mô hình và phần lưu trữ dữ liệu, từ đó tạo điều kiện cho việc triển khai, bảo trì và mở rộng sau này. Cấu trúc này được phản ánh rõ trong repo khi frontend được phục vụ qua giao diện web, backend đảm nhận xác thực và nghiệp vụ, còn dịch vụ AI được triển khai như một thành phần độc lập chuyên suy luận ảnh.

Một điểm quan trọng của bài toán là hệ thống không được xem như công cụ thay thế kết luận y khoa, mà là công cụ hỗ trợ sàng lọc và tham khảo ban đầu. Vì vậy, logic của hệ thống phải nhấn mạnh vai trò hỗ trợ ra quyết định, lưu trữ lịch sử, tiếp nhận phản hồi và không tách rời con người khỏi vòng kiểm soát. Đây cũng là định hướng phù hợp hơn với các ứng dụng AI trong y tế, nơi yêu cầu về độ tin cậy, khả năng theo dõi và cải tiến liên tục luôn quan trọng không kém độ chính xác của mô hình.

3.2. Phân tích yêu cầu hệ thống

Bảng 3.1. Yêu cầu chức năng

Mã	Tác nhân	Mô tả
FR1	Khách	Tải ảnh theo quick-check và nhận dự đoán mà không cần đăng nhập
FR2	Người dùng	Đăng nhập bằng Google, nhận JWT và lưu hồ sơ tài khoản
FR3	Người dùng	Tải một hoặc nhiều ảnh và nhận top-k kết quả phân loại cùng độ tin cậy
FR4	Người dùng	Xem lịch sử dự đoán đã lưu trong tài khoản
FR5	Người dùng	Gửi feedback sau dự đoán để hỗ trợ cải tiến dữ liệu
FR6	Quản trị viên	Truy xuất retrain samples phục vụ vòng lặp cải tiến mô hình
FR7	Hệ thống	Loại bỏ ảnh không phải da và trả về nhãn NON_SKIN_IMAGE
FR8	Hệ thống	Lưu metadata ảnh, xác suất, topKJson và rawResponseJson

Bảng 3.2. Yêu cầu phi chức năng

Mã	Thuộc tính	Mô tả
NFR1	Bảo mật	Xác thực Google + JWT; endpoint công khai và riêng tư được tách rõ trong SecurityConfig
NFR2	Hiệu năng	Tách riêng ModelService để suy luận độc lập, hỗ trợ batch tối đa 32 ảnh/request
NFR3	Độ tin cậy	Lưu rawResponseJson, SHA-256 ảnh và lịch sử prediction để dễ truy vết
NFR4	Khả năng mở rộng	Backend và ModelService có thể thay đổi hoặc triển khai độc lập
NFR5	Khả năng bảo trì	Phân tách controller, service, entity, config và security
NFR6	Độ chính xác	Tối ưu macro-F1 và balanced accuracy

3.3. Phân tích tác nhân của hệ thống

Tác nhân thứ nhất là người dùng. Đây là đối tượng trực tiếp sử dụng hệ thống để tải ảnh, nhận kết quả, xem lịch sử và gửi phản hồi. Người dùng là nguồn phát sinh dữ liệu đầu vào của hệ thống và cũng là đối tượng hưởng lợi từ kết quả phân tích. Các chức năng dành cho người dùng vì vậy phải được tổ chức đơn giản, dễ thao tác và đủ liên tục để hình thành trải nghiệm sử dụng thực tế.

Tác nhân thứ hai là quản trị viên. So với người dùng thông thường, quản trị viên có phạm vi truy cập rộng hơn và chịu trách nhiệm theo dõi hoạt động chung của hệ thống. Quản trị viên cần quan tâm đến số lượng người dùng, kết quả dự đoán, dữ liệu phản hồi, tình trạng ảnh được tải lên và các mẫu có thể được dùng cho quá trình cải thiện mô hình. Vai trò này có ý nghĩa đặc biệt trong các hệ thống AI ứng dụng, vì nó tạo cầu nối giữa quá trình vận hành thực tế và quá trình đánh giá, nâng cấp mô hình.

Tác nhân thứ ba là khối trí tuệ nhân tạo. Dù không phải con người, đây vẫn là một thành phần có thể xem như tác nhân xử lý trong hệ thống, bởi nó đảm nhận vai trò phân tích ảnh và tạo ra kết quả dự đoán. Khối này hoạt động theo yêu cầu từ backend, nhận dữ liệu ảnh, thực hiện suy luận và phản hồi kết quả có cấu trúc. Việc nhận diện rõ vai trò của khối này giúp phân biệt rạch ròi giữa phần nghiệp vụ ứng dụng và phần chuyên trách mô hình.

Tác nhân thứ tư là cơ sở dữ liệu và lớp lưu trữ. Dù về mặt kỹ thuật đây là thành phần hạ tầng, nhưng xét về logic hệ thống, nó đóng vai trò thiết yếu trong việc duy trì trí nhớ vận hành. Mọi dữ liệu người dùng, ảnh tải lên, lịch sử dự đoán và phản hồi đều cần được lưu giữ tại đây để phục vụ truy xuất, thống kê và phân tích trong tương lai.

3.4. Phân tích use case của hệ thống

Use case đầu tiên là đăng nhập và xác thực. Người dùng truy cập hệ thống, thực hiện quá trình đăng nhập, sau đó hệ thống xác nhận danh tính và cho phép tiếp cận các chức năng tương ứng với vai trò của tài khoản. Đây là điểm khởi đầu của mọi tương tác tiếp theo và là tiền đề để dữ liệu sử dụng được gắn với đúng người dùng.

Use case thứ hai là cập nhật hồ sơ cá nhân. Sau khi truy cập hệ thống, người dùng có thể bổ sung các thông tin cần thiết để hoàn thiện hồ sơ. Về mặt nghiệp vụ, việc này giúp hệ thống gắn kết kết quả dự đoán với một hồ sơ người dùng có ý nghĩa hơn thay vì chỉ là một phiên truy cập tạm thời.

Use case thứ ba là tải ảnh và yêu cầu kiểm tra. Đây là use case trung tâm của toàn bộ hệ thống. Người dùng chọn ảnh, gửi yêu cầu lên hệ thống, hệ thống tiếp nhận ảnh, lưu dữ liệu cần thiết, gọi khối AI để suy luận và sau đó trả kết quả về cho

người dùng. Đây là điểm giao thoa rõ nhất giữa công nghệ web, xử lý dữ liệu ảnh và trí tuệ nhân tạo.

Use case thứ tư là xem kết quả dự đoán. Sau khi xử lý, hệ thống hiển thị kết quả chính, mức độ tin cậy và các khả năng liên quan. Người dùng cần có khả năng đọc kết quả theo cách trực quan và dễ hiểu, đồng thời đây cũng là dữ liệu đầu vào cho quyết định có nên phản hồi hay không.

Use case thứ năm là xem lịch sử dự đoán. Người dùng có thể xem lại các lần kiểm tra trước đó để đối chiếu và theo dõi quá trình sử dụng. Về mặt hệ thống, đây là một use case quan trọng vì nó khẳng định rằng kết quả không bị mất sau mỗi phiên làm việc mà được tổ chức thành dữ liệu có thể truy hồi.

Use case thứ sáu là gửi phản hồi. Sau khi xem kết quả, người dùng hoặc quản trị viên có thể đánh giá mức độ phù hợp của dự đoán. Phản hồi này giúp hệ thống thu thập dữ liệu thực tế, hỗ trợ cải thiện chất lượng mô hình và tạo cơ sở cho những lần tái huấn luyện sau này.

Use case thứ bảy là quản trị hệ thống. Quản trị viên truy cập dashboard, quan sát dữ liệu tổng hợp, xem danh sách người dùng, truy xuất lịch sử dự đoán của từng cá nhân, xem ảnh liên quan và kiểm soát phản hồi. Đây là use case phản ánh rõ nhất vai trò vận hành và giám sát của hệ thống ở quy mô lớn hơn mức một người dùng đơn lẻ.

3.5. Kiến trúc tổng thể của hệ thống

Về tổng thể, hệ thống được xây dựng theo kiến trúc nhiều lớp chức năng, trong đó mỗi lớp đảm nhận một nhóm nhiệm vụ tương đối độc lập nhưng có liên kết chặt chẽ trong quá trình vận hành. Đây là cách tổ chức phù hợp cho một bài toán AI ứng dụng, vì nó giảm sự phụ thuộc trực tiếp giữa giao diện, nghiệp vụ, mô hình và dữ liệu.

Lớp đầu tiên là lớp giao diện người dùng. Đây là nơi người dùng và quản trị viên trực tiếp tương tác với hệ thống. Giao diện phải cho phép thao tác đăng nhập, cập nhật hồ sơ, tải ảnh, xem kết quả, xem lịch sử, gửi phản hồi và truy cập khu vực quản trị khi có đủ quyền. Trong project, giao diện web đã được tổ chức với cả khu vực người dùng và khu vực quản trị trên cùng một nền tảng.

Lớp thứ hai là lớp xử lý nghiệp vụ. Lớp này đóng vai trò trung tâm điều phối toàn bộ hệ thống. Mọi yêu cầu từ giao diện đều đi qua lớp này để được xác thực, kiểm tra, lưu trữ, chuyển tiếp và tổng hợp phản hồi. Nói cách khác, đây là lớp chịu trách nhiệm biến các hành động của người dùng thành các thao tác nghiệp vụ có ý nghĩa trong hệ thống. Trong repo, backend đã được tổ chức để đảm nhận rõ các nhóm chức năng như xác thực, quản lý người dùng, dự đoán, phản hồi và quản trị.

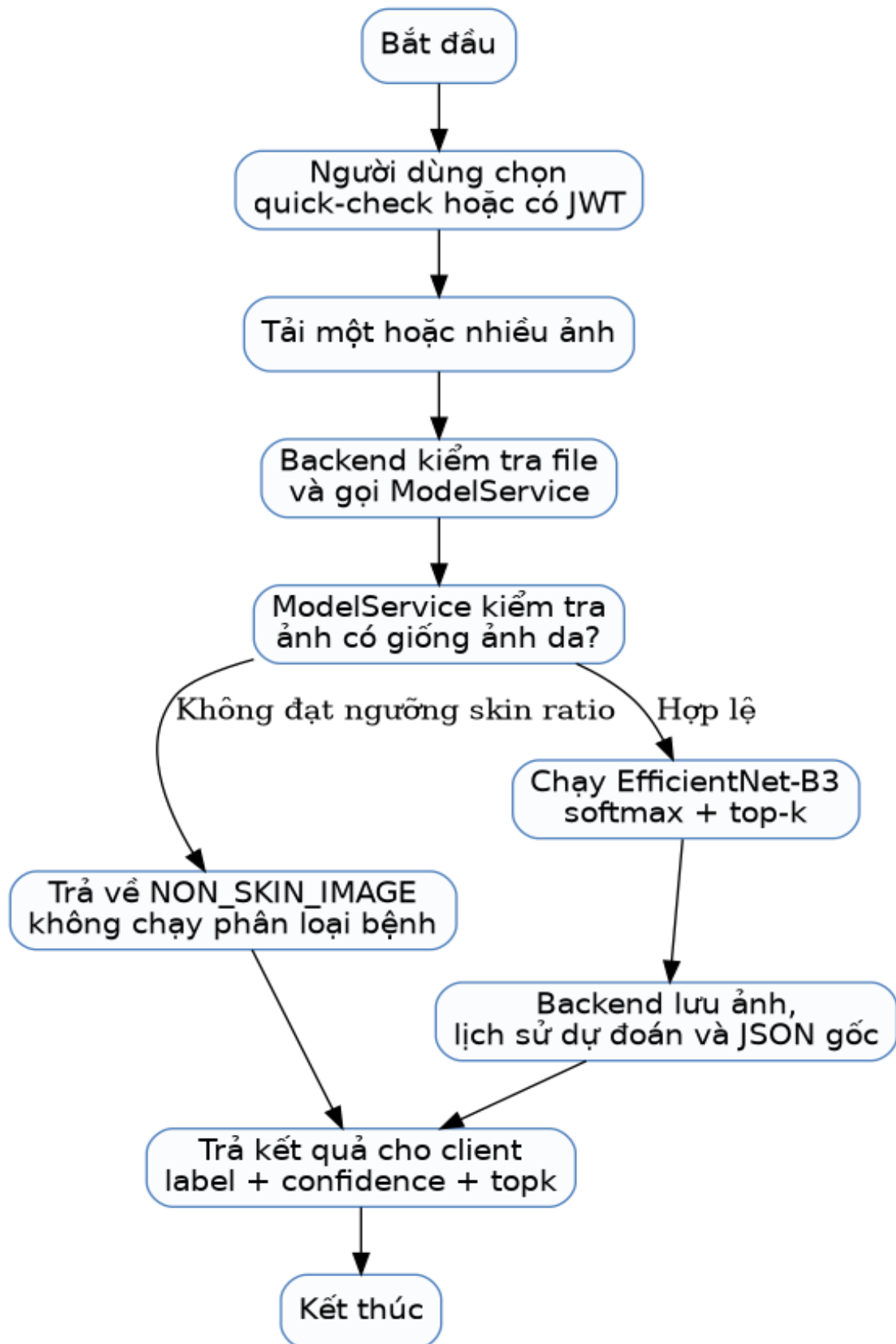
Lớp thứ ba là lớp suy luận trí tuệ nhân tạo. Khối này không quản lý người dùng và cũng không trực tiếp phục vụ giao diện, mà chỉ tập trung vào một nhiệm vụ

chuyên môn là nhận ảnh và trả kết quả phân loại. Cách tách riêng này làm cho kiến trúc trở nên sáng sủa hơn: phần AI có thể được thay đổi, tối ưu hoặc triển khai độc lập mà không ảnh hưởng trực tiếp đến cấu trúc nghiệp vụ chung. Mô hình hiện tại của project được phục vụ bởi một dịch vụ riêng, sử dụng mô hình học sâu để xử lý ảnh và trả về nhãn dự đoán cùng các xác suất liên quan .

Lớp thứ tư là lớp dữ liệu. Đây là nơi lưu trữ thông tin người dùng, ảnh tải lên, lịch sử dự đoán và phản hồi. Nếu không có lớp này, hệ thống chỉ là một công cụ suy luận tức thời và không có khả năng truy hồi, thống kê hay quản trị. Trong project, dữ liệu liên quan đến người dùng, ảnh và dự đoán đã được cấu trúc thành những thực thể độc lập, phản ánh rõ vai trò của lớp dữ liệu trong toàn hệ thống .

Xét ở mức triển khai, hệ thống cũng được tổ chức theo hướng nhiều dịch vụ có thể chạy tách biệt. Cấu hình triển khai trong repo cho thấy phần nghiệp vụ, phần AI và cơ sở dữ liệu được phân chia thành các thành phần riêng, thuận lợi cho việc đóng gói bằng container và triển khai trên hạ tầng đám mây hoặc máy chủ riêng .

3.6. Thiết kế luồng nghiệp



Hình 3.1. Lưu đồ xử lý dự đoán ảnh

3.7. Đánh giá kiến trúc và định hướng phát triển

Kiến trúc hiện tại của hệ thống có ưu điểm ở chỗ phân chia rõ trách nhiệm giữa các thành phần. Giao diện tập trung vào trải nghiệm sử dụng, backend tập trung vào nghiệp vụ, dịch vụ AI tập trung vào suy luận, còn cơ sở dữ liệu tập trung vào lưu trữ. Cách tổ chức này giúp giảm phụ thuộc chéo, thuận lợi cho triển khai và dễ mở rộng. Nếu sau này cần thay mô hình tốt hơn, thêm chức năng thống kê hoặc tích hợp nền tảng triển khai mới, hệ thống vẫn có nền tảng phù hợp để phát triển tiếp.

Bên cạnh đó, kiến trúc hiện tại cũng phản ánh đúng logic của một hệ thống AI ứng dụng trong y tế: dự đoán không phải điểm kết thúc, mà là một mắt xích trong chu trình gồm tiếp nhận dữ liệu, suy luận, lưu trữ, phản hồi và cải tiến. Chính yếu tố này làm cho project có giá trị hơn một mô hình học sâu chạy thử trong môi trường nghiên cứu đơn thuần.

Tuy vậy, trong định hướng phát triển lâu dài, hệ thống vẫn có thể được hoàn thiện thêm theo nhiều hướng như cải tiến khả năng giải thích kết quả, tăng cường kiểm tra chất lượng ảnh đầu vào, mở rộng chức năng báo cáo thống kê, hoặc chuẩn hóa sâu hơn quy trình thu nhận dữ liệu phản hồi phục vụ tái huấn luyện. Những hướng này không làm thay đổi kiến trúc nền tảng, mà chủ yếu làm giàu thêm giá trị ứng dụng và giá trị nghiên cứu của hệ thống.

3.8. Kết luận chương

Chương 3 cho thấy hệ thống của đề tài không chỉ là một mô hình nhận dạng ảnh da liễu, mà là một hệ thống ứng dụng có cấu trúc tương đối hoàn chỉnh, gồm giao diện người dùng, lớp xử lý nghiệp vụ, dịch vụ suy luận mô hình và lớp dữ liệu lưu trữ. Các yêu cầu chức năng và phi chức năng được xác định theo hướng phục vụ đồng thời người dùng, quản trị viên và nhu cầu vận hành thực tế của hệ thống. Kiến trúc được lựa chọn phù hợp với đặc thù của một ứng dụng AI trong y tế, vừa bảo đảm khả năng sử dụng, vừa tạo điều kiện cho việc triển khai, theo dõi và cải tiến về sau.

CHƯƠNG 4: CÀI ĐẶT HỆ THỐNG

4.1. Vai trò của backend trong hệ thống

Trong hệ thống SkinCancer, backend là thành phần đóng vai trò điều phối trung tâm giữa ba miền chức năng chính: giao diện người dùng, cơ sở dữ liệu và dịch vụ suy luận trí tuệ nhân tạo. Nếu giao diện web là nơi người dùng tải ảnh, xem kết quả và gửi phản hồi, còn ModelService là nơi thực hiện suy luận ảnh bằng mô hình học sâu, thì backend là nơi tiếp nhận toàn bộ yêu cầu từ client, áp dụng các quy tắc xác thực, xử lý nghiệp vụ, lưu trữ dữ liệu và phối hợp với dịch vụ suy luận để hình thành phản hồi cuối cùng cho người dùng. Cấu trúc này được thể hiện rõ trong README của dự án, nơi luồng hoạt động được mô tả theo hướng frontend gửi yêu cầu tới backend, backend gọi FastAPI để suy luận rồi lưu kết quả vào cơ sở dữ liệu trước khi trả về giao diện .

Vai trò đầu tiên của backend là thực hiện điều phối nghiệp vụ. Người dùng không làm việc trực tiếp với cơ sở dữ liệu hay dịch vụ mô hình, mà thông qua các endpoint của backend. Khi người dùng đăng nhập, backend kiểm tra danh tính và sinh thông tin xác thực cho phiên làm việc. Khi người dùng tải ảnh, backend nhận file, lưu file trên hệ thống, ghi nhận thông tin ảnh, gọi dịch vụ AI, chuẩn hóa dữ liệu trả về, lưu bản ghi dự đoán và cuối cùng mới gửi response cho client. Khi người dùng truy xuất lịch sử, gửi feedback hay khi quản trị viên truy cập dashboard, backend tiếp tục là thành phần chịu trách nhiệm thực thi các quy tắc nghiệp vụ thích hợp .

Vai trò thứ hai là kiểm soát bảo mật và phân quyền truy cập. Backend hiện tại không dùng session truyền thống mà dùng cơ chế token theo kiểu stateless. Thành phần SecurityConfig cấu hình chỉ cho phép công khai một số route nhất định như route đăng nhập, route health-check và quick-check; còn lại phải xác thực đầy đủ. Những route quản trị còn được bảo vệ thêm bằng kiểm tra vai trò ADMIN ở mức method security . Điều này cho thấy backend không chỉ có vai trò chuyển tiếp dữ liệu mà còn là lớp kiểm soát an toàn của toàn bộ hệ thống.

Vai trò thứ ba là hình thành trí nhớ vận hành của hệ thống. Mọi thông tin như người dùng nào đã đăng nhập, ảnh nào đã được tải lên, kết quả nào đã được mô hình trả ra, xác suất dự đoán là bao nhiêu, người dùng phản hồi ra sao, phản hồi nào được phép dùng cho retrain đều được backend gắn vào các thực thể dữ liệu tương ứng. Nhờ đó, hệ thống không dừng ở mức “chạy xong rồi mất kết quả”, mà có khả năng truy vết, thống kê và tái sử dụng dữ liệu trong những vòng cải tiến tiếp theo. Đây là vai trò rất quan trọng nếu đặt project trong định hướng AI ứng dụng thay vì chỉ là một mô hình nghiên cứu rời rạc.

Vai trò thứ tư là **tách biệt phần nghiệp vụ ứng dụng khỏi phần mô hình AI**. Trong kiến trúc hiện tại, frontend không gọi trực tiếp sang ModelService mà luôn đi qua backend. Cách làm này giúp backend có thể kiểm soát đầu vào, chuẩn hóa dữ liệu, che giấu thông tin nội bộ của dịch vụ mô hình và xử lý lỗi tập trung. Về mặt kiến trúc, đây là một quyết định hợp lý vì nó làm cho hệ thống bền vững hơn khi sau này thay đổi mô hình, đổi địa chỉ service hoặc triển khai trên môi trường khác.

4.2. Kiến trúc tổng quát của backend

Backend của project được xây dựng bằng Spring Boot 3.3.2, sử dụng Java 17, tích hợp các thành phần chính gồm Spring Web, Spring Security, Spring Validation, Spring Data JPA và các thư viện JWT. Phần phụ thuộc trong pom.xml cho thấy backend được thiết kế như một RESTful service có khả năng xử lý web request, bảo vệ API, làm việc với JPA, kết nối PostgreSQL và phát hành JWT để xác thực người dùng. Điều này xác nhận rằng backend không phải là lớp phụ trợ đơn giản, mà là một dịch vụ ứng dụng hoàn chỉnh.

Về mặt tổ chức mã nguồn, backend được phân thành các nhóm lớp chính: nhóm controller chịu trách nhiệm công bố API; nhóm service xử lý nghiệp vụ; nhóm entity mô hình hóa dữ liệu; nhóm repository truy cập cơ sở dữ liệu; nhóm security và config phụ trách xác thực, phân quyền và cấu hình hệ thống. Đây là mô hình tổ chức phổ biến và phù hợp với loại bài toán có nhiều luồng nghiệp vụ độc lập nhưng liên thông với nhau.

Kiến trúc vận hành của backend có thể mô tả theo chuỗi như sau. Đầu tiên, client gửi request tới một endpoint tương ứng. Sau đó, SecurityFilterChain và JwtAuthFilter quyết định request này có cần xác thực không và nếu có thì cố gắng khôi phục người dùng từ JWT. Khi request đã qua lớp bảo mật, controller nhận dữ liệu đầu vào, gọi service thích hợp, service tiếp tục làm việc với repository hoặc gọi sang dịch vụ FastAPI. Kết quả được service chuẩn hóa, đóng gói thành DTO response và trả ngược về controller để gửi ra ngoài. Frontend không gọi trực tiếp sang ModelService mà luôn đi qua backend. Cách làm này giúp backend có thể kiểm soát đầu vào, chuẩn hóa dữ liệu, che giấu thông tin nội bộ của dịch vụ mô hình và xử lý lỗi tập trung. Chuỗi này giúp backend giữ được tính nhất quán: lớp nào làm đúng việc của lớp đó, không ôm quá nhiều trách nhiệm cùng lúc.

Kiến trúc như vậy còn phản ánh đúng đặc thù của project SkinCancer. Hệ thống không chỉ có một use case duy nhất mà có nhiều luồng nghiệp vụ tương đối độc lập: đăng nhập, cập nhật hồ sơ, dự đoán, xem lịch sử, gửi feedback, xem dashboard admin, truy xuất retrain samples. Nếu không chia backend thành các lớp rõ ràng, mã nguồn sẽ dễ bị lẫn lộn giữa logic web, logic bảo mật, logic nghiệp vụ và logic dữ liệu. Với quy mô đồ án hiện tại, cách tổ chức module hóa ở mức này là vừa đủ sâu để báo cáo được, đồng thời vẫn bám sát hiện trạng project.

4.2.1. Lớp giao tiếp API

Lớp giao tiếp API của backend được thể hiện qua nhóm controller. Trong project hiện có các controller tiêu biểu như AuthController, UserController, PredictionController, FeedbackController và AdminController. Việc chia controller theo miền nghiệp vụ là hợp lý vì mỗi nhóm API đại diện cho một cụm chức năng rõ ràng trong hệ thống.

AuthController phụ trách luồng xác thực người dùng. Nó tiếp nhận yêu cầu đăng nhập, nhận token từ phía client và chuyển cho service xử lý xác minh. UserController quản lý các thao tác liên quan đến hồ sơ người dùng, bao gồm xem hồ sơ hiện tại và cập nhật hồ sơ. PredictionController là controller trung tâm cho bài toán của đề tài vì nó xử lý upload ảnh, yêu cầu dự đoán và truy xuất lịch sử dự đoán. FeedbackController cho phép người dùng gắn phản hồi vào một prediction cụ thể và cho admin lấy danh sách retrain samples. AdminController công bố các endpoint dành riêng cho quản trị như dashboard, danh sách user, lịch sử prediction theo user và truy xuất ảnh gốc.

Về mặt thiết kế, controller trong project được giữ tương đối mỏng, tức là không chứa nhiều logic nghiệp vụ phức tạp. Ví dụ PredictionController chỉ nhận MultipartFile, lấy UserPrincipal từ context, đọc một số query param cần thiết rồi chuyển tiếp cho PredictionService xử lý. Đây là lựa chọn thiết kế tốt vì nó giúp controller chỉ đóng vai trò “cửa vào”, còn xử lý nghiệp vụ chi tiết được dồn về service nơi dễ kiểm thử và dễ mở rộng hơn.

Ngoài ra, lớp API còn phải bảo đảm tính nhất quán về giao diện dữ liệu. Các request login dùng DTO riêng, response auth dùng AuthResponse, prediction trả PredictionBatchResponse hoặc PredictionItemResponse, feedback trả FeedbackResponse, admin trả các DTO dashboard và user summary tương ứng. Dù chưa cần phân tích sâu từng DTO trong mục này, cách tổ chức đó cho thấy backend đã phân biệt khá rõ giữa entity dữ liệu nội bộ và dữ liệu public trả ra ngoài.

4.2.2. Lớp xác thực và kiểm soát truy cập

Lớp xác thực của backend được tổ chức quanh các thành phần chính gồm SecurityConfig, JwtAuthFilter, JwtService, UserPrincipal và luồng đăng nhập Google thông qua AuthService cùng GoogleTokenVerifier. Cấu trúc này cho thấy project sử dụng mô hình xác thực lai: đầu vào danh tính được lấy từ Google, còn quyền truy cập nội bộ hệ thống được duy trì bằng JWT do backend phát hành.

SecurityConfig là nơi định nghĩa luật truy cập tổng thể. Theo cấu hình hiện tại, các tài nguyên tĩnh như trang chủ, CSS, JS được mở công khai; endpoint POST /api/auth/google được cho phép không cần xác thực để phục vụ đăng nhập; POST /api/predictions/quick-check cũng được cho phép công khai để phục vụ cơ chế kiểm tra nhanh; trong khi phần lớn các endpoint còn lại yêu cầu người dùng đã được xác

thực . Ngoài ra, AdminController còn được bảo vệ thêm bằng `@PreAuthorize("hasRole('ADMIN')")`, nghĩa là không chỉ cần có token hợp lệ mà còn phải mang vai trò quản trị viên mới truy cập được . Đây là hai tầng bảo vệ bổ sung cho nhau: tầng route-level trong SecurityConfig và tầng method-level trong controller.

JwtAuthFilter là bộ lọc chạy một lần cho mỗi request API. Bộ lọc này đọc header Authorization, kiểm tra xem có theo định dạng Bearer <token> hay không, sau đó đưa token sang JwtService để phân tích. Nếu token hợp lệ, filter tạo UsernamePasswordAuthenticationToken và đặt vào SecurityContextHolder, đồng thời gán quyền dạng ROLE_... tương ứng với role trong token . Cách làm này chuẩn theo cơ chế bảo mật của Spring Security: hệ thống không cần hỏi database ở mỗi request, mà tái dựng người dùng từ JWT, nhờ đó giữ được tính stateless.

JwtService chịu trách nhiệm sinh và parse JWT. Khi sinh token, service đặt subject là userId, thêm hai claim là email và role, sau đó ký bằng secret key HMAC lấy từ cấu hình ứng dụng. Khi parse token, service đọc lại ba thông tin này và ánh xạ thành UserPrincipal nội bộ của hệ thống . Đây là một thiết kế khá gọn và phù hợp: token chứa đủ thông tin để backend xác định người dùng và quyền truy cập mà không phải truy vấn thêm dữ liệu trong phần lớn trường hợp.

Một chi tiết kỹ thuật quan trọng là hệ thống hiện có hai vai trò cụ thể, được định nghĩa bởi enum Role: USER và ADMIN . Vai trò USER dùng cho người dùng thông thường: đăng nhập, cập nhật hồ sơ, upload ảnh, xem lịch sử, gửi feedback. Vai trò ADMIN ngoài các quyền của USER còn có thêm quyền truy cập dashboard, xem danh sách user, xem dự đoán của người khác, truy xuất ảnh và quản lý trạng thái dữ liệu feedback dùng cho retrain. Việc role được đưa trực tiếp vào JWT và ánh xạ thành authority ROLE_USER hoặc ROLE_ADMIN giúp backend xử lý phân quyền nhất quán ở nhiều nơi khác nhau.

4.2.3. Lớp xử lý nghiệp vụ

Lớp xử lý nghiệp vụ được hiện thực qua các service, trong đó nổi bật nhất là AuthService, PredictionService, FeedbackService, UserService và AdminService . Đây là lớp chứa logic cốt lõi của backend, vì toàn bộ quy trình vận hành của hệ thống đều đi qua các service này.

AuthService thực hiện nghiệp vụ đăng nhập. Nó không tự xác minh token Google mà gọi GoogleTokenVerifier, sau đó tìm người dùng theo googleSub trong cơ sở dữ liệu. Nếu chưa có, service tạo người dùng mới với role mặc định là USER, thiết lập createdAt, và tự động gán ADMIN nếu email nằm trong danh sách admin cấu hình. Sau đó, service cập nhật email, họ tên nếu cần, updatedAt, lưu người dùng và sinh JWT để trả về client . Logic này cho thấy đăng nhập không chỉ là xác minh

đánh giá, mà còn là thời điểm đồng bộ hóa hồ sơ Google với thực thể người dùng nội bộ.

PredictionService là service quan trọng nhất về mặt nghiệp vụ. Ở luồng có xác thực, service nhận UserPrincipal, danh sách file, địa chỉ IP client và tham số top-k. Sau đó nó tìm người dùng trong database, lưu ảnh upload vào thư mục vật lý, tạo bản ghi ImageUpload, gọi FastAPI để lấy kết quả suy luận, chuẩn hóa response và tạo ra các bản ghi Prediction tương ứng. Service này còn xử lý cả trường hợp quickPredict, tức là cho phép dự đoán không cần đăng nhập, nhưng không gắn với lịch sử dữ liệu user như luồng đầy đủ. Về mặt thiết kế, đây là service thể hiện rõ nhất việc backend đóng vai trò cầu nối giữa web app và AI service.

FeedbackService xử lý hai nghiệp vụ chính: tạo phản hồi cho một prediction và lấy danh sách retrain samples. Khi tạo feedback, service tìm prediction và user tương ứng, tạo thực thể Feedback, gán các thuộc tính như isCorrect, userLabel, comment, allowForRetrain, createdAt, rồi lưu vào database. Khi lấy retrain samples, service chỉ lấy các feedback có allowForRetrain = true, rồi ánh xạ ra response chuyên biệt để admin có thể dùng cho giai đoạn cải tiến mô hình. Đây là phần rất đáng chú ý vì nó cho thấy hệ thống có cơ chế hậu kiểm và thu thập dữ liệu cho vòng lặp cải tiến tiếp theo.

Ở góc độ kiến trúc, việc đặt các logic này vào service thay vì controller là hợp lý. Điều đó làm cho controller không bị phình to, đồng thời tạo ra nơi tập trung để thực thi transaction, gọi repository, xử lý ngoại lệ và log luồng nghiệp vụ. Với yêu cầu của một đồ án AI ứng dụng, đây là một điểm mạnh quan trọng của backend.

4.2.4. Lớp truy cập dữ liệu

Lớp truy cập dữ liệu của backend được xây dựng theo Spring Data JPA, kết hợp với các entity ánh xạ quan hệ xuống PostgreSQL. Cấu hình datasource trong application.yml cho thấy backend dùng PostgreSQL làm hệ quản trị cơ sở dữ liệu mặc định, với driver org.postgresql.Driver, cấu hình ddl-auto đọc từ biến môi trường và chiến lược đặt tên vật lý/ảnh danh được chuẩn hóa rõ ràng. Việc dùng JPA trong project là hợp lý vì dữ liệu có cấu trúc quan hệ rõ ràng giữa người dùng, ảnh, dự đoán và phản hồi.

Ở mức mô hình dữ liệu, backend có bốn entity cốt lõi là UserEntity, ImageUpload, Prediction và Feedback. Chúng tương ứng trực tiếp với ERD mà bạn đã dựng, cho thấy thiết kế dữ liệu trong code và thiết kế khái niệm ở mức báo cáo là nhất quán với nhau.

UserEntity lưu thông tin người dùng như UserId, GoogleSub, Email, FullName, Gender, Age, Role, CreatedAt, UpdatedAt. Đáng chú ý là bảng được đặt tên "User" có bao ngoặc kép để tránh xung đột với từ khóa SQL. Đây là một chi tiết

kỹ thuật đáng nêu trong báo cáo vì nó cho thấy thiết kế dữ liệu đã tính đến khả năng xung đột tên bảng.

ImageUpload là thực thể đại diện cho mỗi ảnh được người dùng tải lên. Bảng này gắn với UserEntity qua quan hệ nhiều-một, và lưu các thông tin như FileUri, FileHashSha256, FileSizeBytes, UploadedAt . Việc lưu hash SHA-256 giúp hệ thống có thể dùng về sau cho kiểm tra trùng lặp, kiểm soát dữ liệu hoặc truy vết file.

Prediction lưu kết quả suy luận tương ứng với mỗi ảnh đã upload. Nó liên kết tới ImageUpload qua khóa ngoại ImageId, đồng thời lưu các trường như RequestedAt, ClientApp, ClientIp, ModelName, ModelVersion, PredictedClass, Probability, TopKJson, RawResponseJson . Đây là một thiết kế khá đầy đủ, vì nó không chỉ lưu nhãn dự đoán cuối cùng mà còn lưu cả thông tin mô hình và dữ liệu JSON chi tiết, cho phép truy xuất lại ngữ cảnh suy luận nếu cần.

Feedback là thực thể gắn với Prediction và UserEntity, lưu phản hồi hậu kiểm gồm IsCorrect, UserLabel, Comment, AllowForRetrain, CreatedAt . Quan hệ này rất quan trọng vì nó cho phép một prediction được đánh giá bởi người dùng và tiếp tục trở thành nguồn dữ liệu cho quá trình cải thiện mô hình sau này.

Nhìn chung, lớp dữ liệu của backend được thiết kế khá phù hợp với bài toán. Nó phản ánh đúng luồng nghiệp vụ: người dùng tạo ảnh upload, ảnh tạo ra prediction, prediction có thể nhận feedback. Đây là chuỗi quan hệ hợp lý và bám sát logic thực của hệ thống.

4.2.5. Lớp cấu hình và quan sát hệ thống

Lớp cấu hình của backend được tập trung trong application.yml và các properties class liên quan, cho phép hệ thống điều chỉnh hành vi qua biến môi trường thay vì hard-code trong mã nguồn. Các nhóm cấu hình chính bao gồm datasource, JPA, multipart upload, cổng chạy server, JWT, địa chỉ FastAPI, email admin, thư mục upload, cấu hình frontend và CORS . Đây là cách thiết kế cần thiết để backend có thể chạy trong môi trường local, Docker hoặc cloud mà không phải sửa source code.

Trong phần bảo mật, backend dùng app.jwt.secret và app.jwt.expiration-minutes để điều khiển cơ chế JWT. Ở phần tích hợp AI, app.fastapi.base-url và app.fastapi.default-top-k quyết định nơi backend sẽ gửi ảnh sang suy luận và số lượng kết quả top-k mặc định. Ở phần lưu trữ, app.upload.root-dir xác định nơi file ảnh được ghi xuống ổ đĩa. Việc tách rõ các nhóm cấu hình như vậy giúp project dễ triển khai hơn và cũng là điểm nên nêu trong báo cáo khi phân tích thiết kế backend.

Về quan sát hệ thống, một chi tiết đáng chú ý là nhiều service và filter có log nghiệp vụ khá rõ, ví dụ JwtAuthFilter, AuthService, PredictionService, FeedbackService đều ghi log theo flow cụ thể như [FLOW][AUTH], [FLOW][PREDICT], [FLOW][FEEDBACK] hoặc [API][REQ], [API][JWT],

[API][RES] . Điều này cho thấy backend đã có tư duy theo dõi luồng chạy chứ không chỉ viết mã cho đủ chức năng. Trong báo cáo, bạn có thể xem đây là một phần của thiết kế quan sát hệ thống ở mức ứng dụng.

4.3. Thiết kế module xác thực

4.3.1. Xác thực người dùng bằng JWT

Module xác thực của backend được xây dựng theo mô hình token-based authentication. Sau khi người dùng đăng nhập thành công, backend không lưu session trên server theo kiểu truyền thống mà phát hành một JWT chứa danh tính và vai trò của người dùng. Từ đó, mọi request tiếp theo chỉ cần gửi token trong header Authorization: Bearer ..., backend sẽ tự phục hồi thông tin người dùng từ token này mà không cần tạo session state phía server .

Cụ thể, JwtService.generateToken sinh token bằng cách đặt subject là userId, thêm claim email và claim role, sau đó ký bằng secret key HMAC lấy từ cấu hình app.jwt.secret. Token cũng có thời điểm phát hành và thời điểm hết hạn dựa trên expirationMinutes . Đây là một thiết kế phù hợp vì ba thông tin đó là đủ để backend xác định người dùng đang là ai và đang có quyền gì.

Khi request đi vào hệ thống, JwtAuthFilter đọc header Authorization. Nếu header hợp lệ và theo đúng tiền tố Bearer , filter lấy phần token phía sau, gọi jwtService.parse(token) để phân tích claim, sau đó tạo UsernamePasswordAuthenticationToken và đưa vào SecurityContextHolder . Nhờ đó, các controller và service có thể truy cập UserPrincipal của người dùng mà không cần tự viết lại logic xác thực ở mỗi endpoint.

Về mặt thiết kế báo cáo, nên nhấn mạnh rằng JWT ở đây không chỉ có ý nghĩa kỹ thuật, mà còn có ý nghĩa kiến trúc. Nó giúp backend giữ tính stateless, phù hợp với triển khai containerized và thuận lợi hơn khi mở rộng số lượng instance so với session truyền thống.

4.3.2. Xác thực liên kết với Google ID Token

Backend hiện không cho người dùng đăng ký tài khoản bằng form nội bộ, mà sử dụng Google như nhà cung cấp danh tính ban đầu. AuthController nhận idToken từ phía client và chuyển cho AuthService.loginWithGoogle Google Token Verifie dùng RestTemplate gọi end point để xác minh token. Nếu token không hợp lệ, thiếu email hoặc email chưa được Google xác nhận, hệ thống ném ra lỗi không cho đăng nhập .

Sau khi Google token được xác thực, backend lấy các thông tin như sub, danh Google ổn định nhất, nên project dùng googleSub làm khóa tìm người dùng nội bộ. Đây là quyết định thiết kế tốt hơn so với dùng email làm khóa chính, vì email vẫn có thể thay đổi theo thời gian còn sub thì ổn định hơn trong hệ sinh thái Google.

Nếu người dùng chưa tồn tại, AuthService khởi tạo UserEntity mới, gán Role.USER, createdAt và đồng bộ các thông tin như email, tên hiển thị. Nếu email thuộc tập email quản trị đã cấu hình, service nâng role thành ADMIN. Sau đó hệ thống phát hành JWT nội bộ của riêng backend. Điều này có nghĩa là Google chỉ chịu trách nhiệm xác minh danh tính đầu vào, còn quyền truy cập nội bộ vẫn do backend quyết định và duy trì.

4.3.3. Phân quyền theo role

Cơ chế phân quyền của project xoay quanh enum Role với hai giá trị USER và ADMIN. Đây là mô hình role đơn giản nhưng phù hợp với quy mô và yêu cầu của hệ thống. USER là vai trò mặc định, dùng cho mọi tài khoản sau khi đăng nhập lần đầu. ADMIN là vai trò đặc biệt, được gán dựa trên danh sách email quản trị cấu hình ở server.

Về mặt nghiệp vụ, USER có quyền sử dụng các chức năng cá nhân như xem hồ sơ, cập nhật hồ sơ, tải ảnh, xem lịch sử và gửi feedback. ADMIN ngoài các quyền trên còn có quyền truy cập dashboard, danh sách user, xem lịch sử dự đoán của user khác, xem ảnh qua route admin và cập nhật trạng thái feedback dùng cho retrain thông qua AdminController.

Ở tầng kỹ thuật, role được nhúng trong JWT dưới claim role, sau đó JwtAuthFilter chuyển thành SimpleGrantedAuthority("ROLE_" + principal.role().name()). Đây là lý do vì sao trong controller có thể dùng @PreAuthorize("hasRole('ADMIN')"). Cách này bám đúng convention của Spring Security và làm cho việc phân quyền trở nên rõ ràng trong code lẫn trong báo cáo thiết kế.

4.3.4. Bảo vệ route và phạm vi công khai

Một điểm cần nêu rõ trong báo cáo là không phải mọi route đều bị khóa hoàn toàn. SecurityConfig cho phép công khai một số endpoint như tài nguyên tĩnh, /api/health, /api/auth/google và /api/predictions/quick-check. Thiết kế này phản ánh chủ đích nghiệp vụ: người dùng có thể trải nghiệm kiểm tra nhanh mà không cần đăng nhập, nhưng nếu muốn lưu lịch sử, dùng profile hay gửi feedback gắn với tài khoản thì phải xác thực đầy đủ.

Cách thiết kế phân tầng quyền như vậy là hợp lý. Nó vừa hỗ trợ trải nghiệm người dùng mới, vừa giữ được tính đầy đủ của các tính năng khi đã đăng nhập. Đây là chi tiết nên viết rõ vì nó cho thấy backend không chỉ thiết kế về mặt bảo mật mà còn cân bằng với trải nghiệm sử dụng.

4.4. Bảng API và endpoint hệ thống

Bảng 4.1. Cấu trúc bảng API chính của hệ thống

Phương	Endpoint	Quyền truy	Chức năng
--------	----------	------------	-----------

thức		cấp	
POST	/api/auth/google	Public	Đăng nhập bằng Google và nhận JWT
POST	/api/predictions/quick-check	Public	Dự đoán nhanh không cần đăng nhập
POST	/api/predictions	Authenticated	Upload ảnh và lưu lịch sử dự đoán
GET	/api/predictions	Authenticated	Xem lịch sử dự đoán theo trang
POST	/api/predictions/{id}/feedbacks	Authenticated	Gửi feedback sau dự đoán
GET	/api/predictions/retrain-samples	ADMIN	Xuất các mẫu phục vụ tái huấn luyện
GET	/health	Internal / public deploy	Kiểm tra trạng thái ModelService
POST	/v1/predict	Backend -> FastAPI	Suy luận ảnh và trả top-k

4.4.1. Controller xác thực

Controller xác thực được tổ chức dưới tiền tố /api/auth. Trong backend hiện tại, endpoint trọng tâm là POST /api/auth/google, tiếp nhận Google ID token từ phía client, sau đó chuyển yêu cầu sang tầng nghiệp vụ để xác minh danh tính, ánh xạ người dùng vào hệ thống nội bộ và phát hành JWT. Kết quả trả về là AuthResponse, trong đó chứa access token, userId, email, role và trạng thái hoàn thiện hồ sơ người dùng. Đây là điểm bắt đầu của toàn bộ luồng xác thực trong hệ thống, vì mọi chức năng cần bảo vệ phía sau đều dựa trên token được cấp từ bước này.

Thiết kế controller xác thực như vậy là phù hợp với kiến trúc của project vì backend không triển khai cơ chế tài khoản nội bộ theo kiểu truyền thống. Hệ thống không có quy trình đăng ký bằng email và mật khẩu, không có chức năng băm mật khẩu, cũng không có quy trình quên mật khẩu hay đặt lại mật khẩu như các ứng dụng quản trị tài khoản thông thường. Thay vào đó, bước xác thực danh tính ban đầu được giao cho Google, còn backend chỉ tiếp nhận kết quả xác minh đó để xây dựng danh tính nội bộ và kiểm soát quyền truy cập trong phạm vi hệ thống.

Xét ở góc độ kỹ thuật, controller này giữ vai trò là điểm vào của module xác thực, nhưng không trực tiếp xử lý toàn bộ logic đăng nhập. Phần xác minh token Google được thực hiện bởi service chuyên trách, sau đó backend kiểm tra xem

người dùng đã tồn tại hay chưa, gán vai trò mặc định hoặc vai trò quản trị tùy theo cấu hình email admin, đồng bộ thông tin hồ sơ cơ bản rồi mới phát hành JWT cho client. Cách tổ chức đó giúp controller giữ được sự gọn gàng, còn các bước nghiệp vụ phức tạp được chuyển xuống tầng service, nhờ đó thiết kế rõ ràng hơn và phù hợp với cách tổ chức backend theo Spring Boot.

Một điểm đáng chú ý khác là controller xác thực không chỉ có ý nghĩa ở mức đăng nhập, mà còn là nơi liên kết giữa danh tính bên ngoài và hệ thống người dùng nội bộ. Sau khi người dùng đăng nhập thành công, những thông tin như userId, role và trạng thái hoàn thiện hồ sơ sẽ được dùng cho toàn bộ các controller phía sau. Do đó, controller xác thực chính là nền tảng để hình thành cơ chế truy cập có kiểm soát cho toàn bộ hệ thống SkinCancer.

4.4.2. Controller người dùng

Controller người dùng được đặt dưới tiền tố /api/me, phản ánh rõ định hướng thiết kế là mọi thao tác trong nhóm này đều gắn với chính người dùng hiện tại, không phải với một tài khoản bất kỳ được truyền từ phía client. Trong backend hiện có, nhóm endpoint quan trọng nhất gồm GET /api/me để lấy hồ sơ hiện tại và PUT /api/me/profile để cập nhật hồ sơ cá nhân. Các trường dữ liệu được cập nhật chủ yếu là họ tên, giới tính và tuổi, tức là những thông tin phục vụ cả hiển thị giao diện lẫn quản lý dữ liệu nghiệp vụ trong hệ thống.

Thiết kế của controller này mang tính cá nhân hóa rõ rệt. Thay vì để client gửi userId trong request rồi backend mới kiểm tra, hệ thống lấy trực tiếp thông tin người dùng từ UserPrincipal trong ngữ cảnh bảo mật. Điều này có ý nghĩa rất quan trọng về mặt an toàn thiết kế, vì nó loại bỏ khả năng client cố tình giả mạo định danh của người khác khi gọi API. Nói cách khác, controller người dùng được xây dựng theo nguyên tắc “backend tự biết ai đang gọi”, thay vì “client khai mình là ai”.

Ở góc độ nghiệp vụ, controller người dùng có vai trò duy trì hồ sơ nội bộ sau khi xác thực bằng Google. Việc đăng nhập qua Google chỉ cung cấp một số thông tin định danh ban đầu như email, tên hiển thị và định danh sub. Tuy nhiên, hệ thống SkinCancer còn cần thêm các thông tin như giới tính và tuổi để hồ sơ người dùng đầy đủ hơn. Vì vậy, controller này đóng vai trò như cầu nối giữa danh tính đăng nhập và dữ liệu hồ sơ phục vụ sử dụng lâu dài.

Một điểm thiết kế hợp lý là controller người dùng không ôm luôn các chức năng quản trị hay lịch sử dự đoán. Việc tách riêng nhóm thao tác hồ sơ sang một controller độc lập giúp backend rõ ràng hơn về trách nhiệm. Khi nhìn vào cấu trúc endpoint, người đọc có thể phân biệt ngay đâu là thao tác liên quan đến tài khoản cá nhân, đâu là thao tác liên quan đến prediction, và đâu là thao tác dành cho quản trị viên. Đây là cách tổ chức phù hợp với backend theo phong cách module hóa.

4.4.3. Controller dự đoán và lịch sử

Controller dự đoán là thành phần trung tâm nhất của backend xét theo giá trị nghiệp vụ cốt lõi của đề tài. Controller này được đặt dưới tiền tố `/api/predictions`, thể hiện rõ đây là không gian endpoint dành cho việc gửi ảnh, nhận kết quả phân loại, và truy xuất lại các dữ liệu prediction đã được lưu trước đó. Trong project hiện tại, controller này hỗ trợ cả luồng có xác thực lẫn luồng không xác thực. Cụ thể, `POST /api/predictions` hoặc `POST /api/predictions/check` được dùng cho dự đoán có gắn với tài khoản người dùng; còn `POST /api/predictions/quick-check` được dùng cho trường hợp kiểm tra nhanh mà không cần đăng nhập.

Việc hỗ trợ hai kiểu dự đoán trong cùng một controller là hoàn toàn hợp lý về mặt kiến trúc. Cả hai đều có cùng bản chất nghiệp vụ là nhận ảnh đầu vào và trả ra kết quả phân loại, nhưng khác nhau ở ngữ cảnh sử dụng và phạm vi lưu trữ. Luồng có xác thực cho phép backend lưu ảnh, lưu prediction và gắn kết quả với tài khoản người dùng, từ đó hình thành lịch sử cá nhân. Luồng quick-check thì nghiêng về trải nghiệm dùng thử hoặc kiểm tra nhanh, nơi người dùng chỉ cần kết quả tức thời mà không quan tâm đến lưu vết lâu dài. Cách tách endpoint như vậy cho thấy backend đã nghĩ đến cả trải nghiệm tiếp cận ban đầu lẫn quy trình sử dụng đầy đủ.

Xét ở mức thiết kế endpoint, controller này tiếp nhận ảnh dưới dạng `multipart/form-data`, hỗ trợ danh sách file chứ không chỉ một file duy nhất. Điều đó phản ánh khả năng xử lý nhiều ảnh trong một lần yêu cầu, đồng thời phù hợp với logic của `ModelService` vốn cũng hỗ trợ batch inference. Ngoài ra, controller còn nhận tham số `topK`, cho phép điều chỉnh số lượng kết quả xác suất cao nhất mà hệ thống trả về. Đây là chi tiết có giá trị thực tiễn vì trong một số tình huống, người dùng hoặc quản trị viên không chỉ quan tâm đến nhãn đứng đầu mà còn muốn xem thêm các khả năng gần nhất.

Bên cạnh luồng tạo prediction, controller này còn phụ trách truy xuất lịch sử dự đoán thông qua `GET /api/predictions` hoặc `GET /api/predictions/history`, có hỗ trợ phân trang bằng `page` và `size`. Việc đặt lịch sử trong cùng controller với prediction là hợp lý vì đây không phải là một miền nghiệp vụ tách rời, mà là giai đoạn sau của cùng một vòng đời dữ liệu. Prediction được tạo ra từ ảnh tải lên, sau đó được lưu lại và có thể truy xuất như lịch sử. Nếu tách phần history sang một controller hoàn toàn khác, cấu trúc backend có thể trở nên phân mảnh không cần thiết.

Về mặt thiết kế tổng thể, controller dự đoán là nơi thể hiện rõ nhất quan điểm kiến trúc của project: giao diện không giao tiếp trực tiếp với mô hình AI, mà mọi thao tác đều đi qua backend. Nhờ vậy, backend có thể kiểm soát đầu vào, lưu metadata của ảnh, lưu prediction vào cơ sở dữ liệu và xử lý lỗi tập trung nếu dịch vụ AI gặp sự cố. Đây là lựa chọn đúng cho một hệ thống AI ứng dụng, nhất là trong

miền y tế, nơi việc lưu vết và kiểm soát dữ liệu quan trọng không kém bản thân kết quả mô hình.

4.4.4. Controller phản hồi

Controller phản hồi được tổ chức trong cùng không gian `/api/predictions`, thể hiện rõ rằng phản hồi của người dùng không tồn tại như một tài nguyên độc lập, mà luôn gắn với một kết quả prediction cụ thể. Endpoint chính của nhóm này là `POST /api/predictions/{predictionId}/feedback` hoặc `POST /api/predictions/{predictionId}/feedbacks`, cho phép người dùng gửi đánh giá đối với kết quả mà hệ thống đã trả ra trước đó. Ngoài ra, controller còn cung cấp `GET /api/predictions/retrain-samples` dành cho quản trị viên để lấy tập các phản hồi được cho phép sử dụng vào quy trình tái huấn luyện.

Cách thiết kế đưa `predictionId` vào path là hoàn toàn đúng với bản chất nghiệp vụ. Phản hồi không thể xuất hiện nếu không có prediction trước đó, và mọi nội dung trong phản hồi như `isCorrect`, `userLabel`, `comment` hay `allowForRetrain` đều nhằm mô tả chất lượng của một kết quả dự đoán đã tồn tại. Vì vậy, biểu diễn quan hệ này ngay ở mức endpoint giúp API rõ ràng hơn, đồng thời phản ánh chính xác quan hệ khóa ngoại giữa bảng `Feedback` và bảng `Prediction` trong cơ sở dữ liệu.

Ở góc độ vận hành hệ thống, controller phản hồi có ý nghĩa rất lớn vì nó tạo ra tầng hậu kiểm cho mô hình AI. Nếu backend chỉ dừng ở việc nhận ảnh và trả nhãn, hệ thống sẽ chỉ là một công cụ suy luận một chiều. Khi có thêm controller phản hồi, người dùng có thể tham gia đánh giá kết quả, từ đó backend bắt đầu thu thập dữ liệu phục vụ việc cải tiến mô hình. Đây là điểm làm cho project có chiều sâu hơn một bài demo AI thông thường.

Endpoint `GET /api/predictions/retrain-samples` cũng rất đáng chú ý về mặt thiết kế. Nó cho thấy feedback không chỉ để hiển thị hay thống kê, mà còn được quản trị theo hướng có thể chọn lọc những mẫu đủ điều kiện cho các vòng huấn luyện tiếp theo. Tức là controller phản hồi không chỉ phục vụ nghiệp vụ tương tác người dùng, mà còn kết nối trực tiếp với định hướng phát triển mô hình về sau. Đối với một đề án học tập có yếu tố AI ứng dụng, đây là một điểm mạnh đáng nêu trong báo cáo.

4.4.5. Controller quản trị

Controller quản trị được đặt dưới tiền tố `/api/admin` và được bảo vệ ở mức lớp bằng `@PreAuthorize("hasRole('ADMIN')")`. Điều này có nghĩa toàn bộ các endpoint trong controller chỉ được truy cập khi người dùng đã xác thực hợp lệ và mang vai trò quản trị viên. Trong cấu trúc hiện tại của backend, đây là controller đại diện cho tầng vận hành hệ thống, khác với các controller còn lại vốn chủ yếu phục vụ trực tiếp cho người dùng cuối.

Nhóm endpoint quản trị bao gồm các chức năng chính như lấy dashboard tổng quan, lấy danh sách người dùng, xem prediction theo từng người dùng, truy xuất ảnh gốc và điều chỉnh trạng thái phản hồi liên quan đến retrain. Cụ thể, /dashboard cho phép tổng hợp số liệu toàn cục; /users hỗ trợ truy xuất danh sách người dùng; /users/{userId}/predictions cho phép xem lịch sử prediction của một tài khoản cụ thể; /images/{imageId} giúp quản trị viên mở ảnh thực tế đã được tải lên; còn /predictions/{predictionId}/feedback phục vụ việc cập nhật phản hồi liên quan đến dữ liệu retrain.

Thiết kế controller quản trị theo hướng gom các chức năng điều hành vào một không gian endpoint riêng là rất phù hợp. Thứ nhất, nó làm rõ ranh giới giữa backend phục vụ người dùng và backend phục vụ vận hành nội bộ. Thứ hai, nó thuận tiện cho việc áp dụng cơ chế bảo mật, vì chỉ cần bảo vệ controller ở mức role ADMIN là đã có thể chặn toàn bộ nhóm chức năng nhạy cảm. Thứ ba, nó cho phép mở rộng các chức năng quản trị trong tương lai mà không làm rối nhóm endpoint người dùng thông thường.

Về mặt học thuật, controller quản trị là một minh chứng rõ ràng rằng hệ thống AI của đề tài không chỉ được xây dựng để demo dự đoán, mà còn có chu trình theo dõi, giám sát và quản lý dữ liệu thực tế. Một hệ thống AI muốn tiến tới ứng dụng lâu dài thì không thể thiếu tầng quản trị như vậy, bởi phải có nơi để quan sát tình trạng người dùng, thống kê kết quả, kiểm tra ảnh và rà soát chất lượng phản hồi. Vì vậy, sự hiện diện của controller quản trị làm tăng đáng kể giá trị hệ thống của project.

4.4.6. Nhận xét về cách tổ chức controller và endpoint

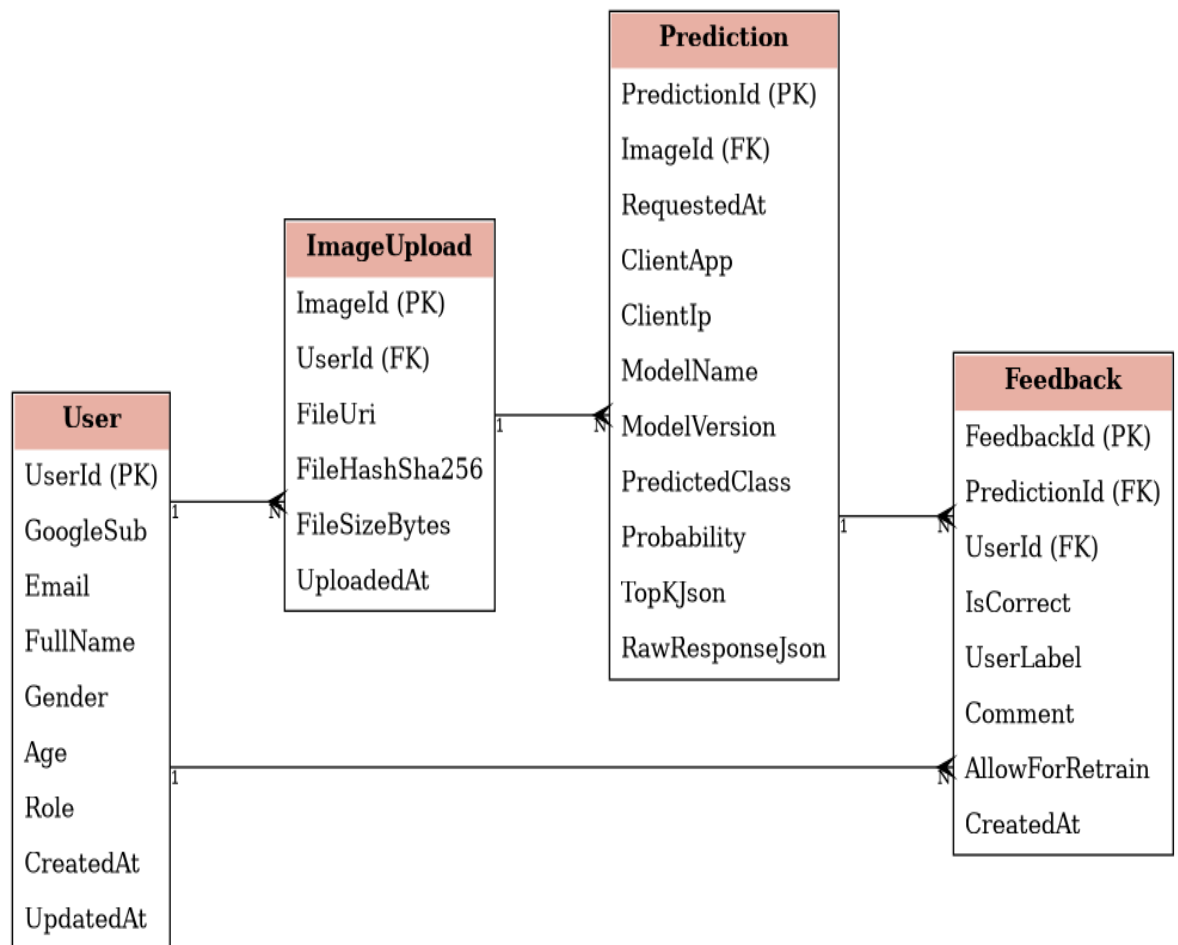
Nhìn tổng thể, việc tổ chức backend thành các controller xác thực, người dùng, dự đoán, phản hồi và quản trị là hợp lý với kiến trúc Spring Boot hiện tại. Cách tổ chức này vừa bám sát mã nguồn thực tế của project, vừa giúp các endpoint được chia theo miền nghiệp vụ rõ ràng. So với cách gọi “router”, cách dùng thuật ngữ “controller” phù hợp hơn với công nghệ của backend, vì các endpoint thực tế đang được công bố thông qua các lớp @RestController và các annotation như @RequestMapping, @GetMapping, @PostMapping, @PutMapping, @PatchMapping.

Ngoài việc đúng với stack công nghệ, cách viết theo controller còn giúp báo cáo chặt chẽ hơn ở góc độ thiết kế. Khi trình bày controller, có thể liên hệ trực tiếp tới từng lớp mã nguồn, từng nhóm endpoint và từng luồng nghiệp vụ tương ứng. Điều này giúp phần Chương 4 vừa có tính kỹ thuật, vừa có tính đối chiếu rõ ràng giữa báo cáo và hiện trạng triển khai của project.

Tóm lại, trong bối cảnh project SkinCancer sử dụng Spring Boot, cách gọi đúng và chuẩn hơn là **thiết kế controller và endpoint**, thay vì **thiết kế router và**

endpoint. Việc chuyển sang cách gọi này giúp nội dung báo cáo chính xác hơn, sát hơn với mã nguồn và chuyên nghiệp hơn về mặt học thuật.

4.4.7. Sơ đồ quan hệ thực thể ERD



Hình 4.1. Sơ đồ quan hệ thực thể ERD

4.5. Cơ sở dữ liệu

Bảng 4.2. Cấu trúc bảng User

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
UserId	UUID	PK, Not null	Định danh duy nhất của người dùng trong hệ thống.
GoogleSub	varchar(100)	Unique, Not null	Mã định danh người dùng do Google cung cấp sau khi xác thực.
Email	varchar(256)	Not null	Địa chỉ email dùng để đăng nhập và hiển thị thông tin tài khoản.
FullName	varchar(150)	Nullable	Họ và tên hiển thị của người dùng.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
Gender	nvarchar(10)	Nullable	Giới tính người dùng phục vụ hoàn thiện hồ sơ.
Age	int	Nullable	Tuổi của người dùng.
Role	varchar(10)	Not null	Vai trò truy cập của người dùng, gồm USER hoặc ADMIN.
CreatedAt	timestamp(6)	Not null	Thời điểm tài khoản được tạo trong hệ thống.
UpdatedAt	timestamp(6)	Nullable	Thời điểm hồ sơ người dùng được cập nhật gần nhất.

Bảng 4.3. Cấu trúc bảng ImageUpload

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
ImageId	UUID	PK, Not null	Định danh duy nhất của ảnh được tải lên.
UserId	UUID	FK -> User.UserId, Not null	Khóa ngoại liên kết ảnh với người dùng sở hữu.
FileUri	char(500)	Not null	Đường dẫn lưu trữ ảnh trên hệ thống tệp hoặc đường dẫn logic tương ứng.
FileHashSha256	char(64)	Nullable	Giá trị băm SHA-256 của nội dung ảnh để kiểm tra trùng lặp và toàn vẹn dữ liệu.
FileSizeBytes	bigint	Nullable	Kích thước của file ảnh tính theo byte.
UploadedAt	timestamp(6)	Not null	Thời điểm ảnh được người dùng tải lên hệ thống.

Bảng 4.4. Cấu trúc bảng Prediction

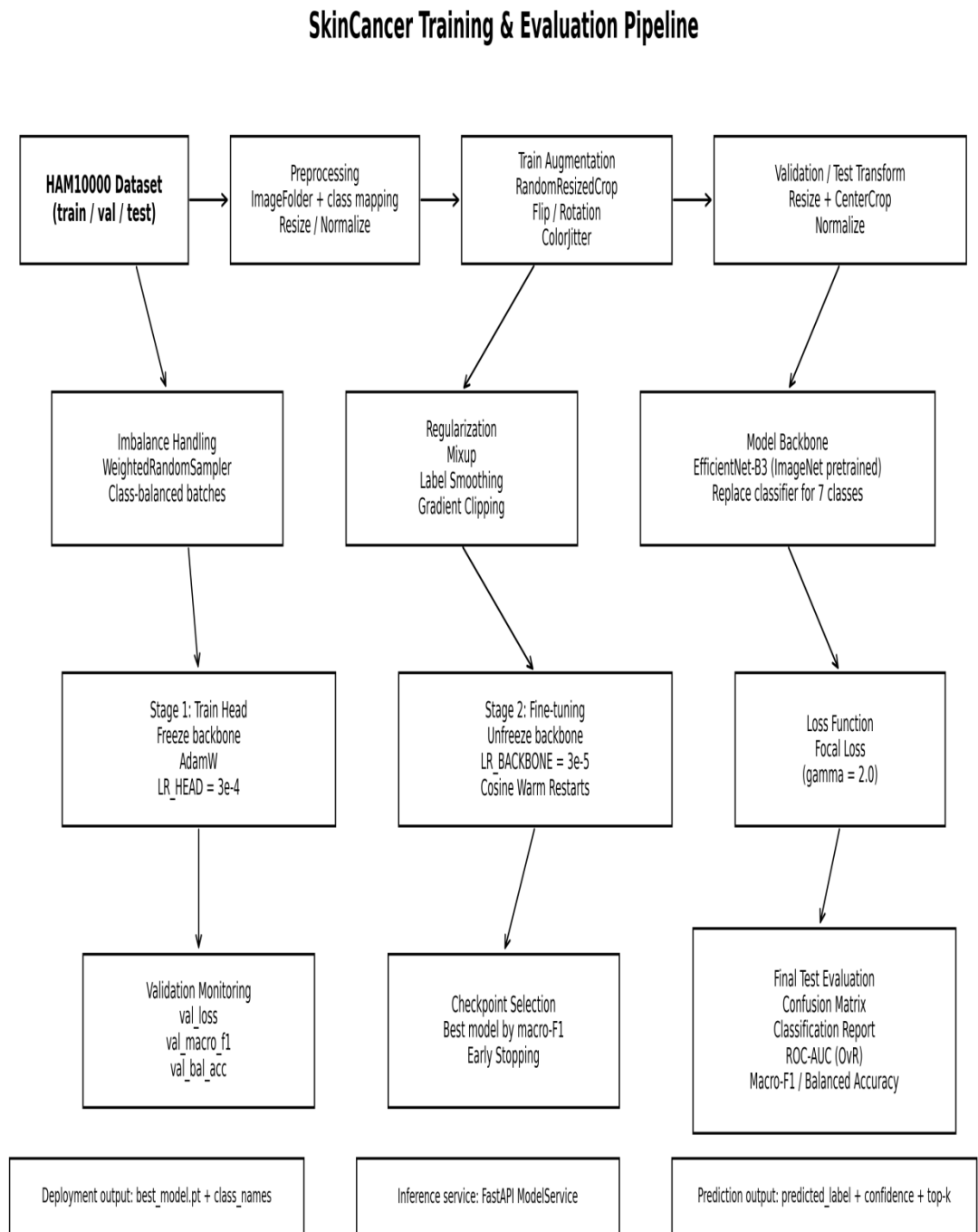
Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
PredictionId	UUID	PK, Not null	Định danh duy nhất của kết quả dự đoán.
ImageId	UUID	FK -> ImageUpload. ImageId, Not null	Khóa ngoại liên kết với ảnh đầu vào được dùng để suy luận.
RequestedAt	timestamp(6)	Not null	Thời điểm backend ghi nhận yêu cầu dự đoán.
ClientApp	varchar(50)	Nullable	Nguồn client hoặc ứng dụng gọi API.
ClientIp	varchar(50)	Nullable	Địa chỉ IP của request gửi tới backend.
ModelName	varchar(100)	Nullable	Tên mô hình dùng để tạo ra kết quả dự đoán.
ModelVersion	varchar(50)	Nullable	Phiên bản của mô hình tại thời điểm suy luận.
PredictedClass	varchar(100)	Not null	Lớp dự đoán chính được hệ thống trả về.
Probability	decimal(6,5)	Not null	Xác suất của lớp dự đoán chính.
TopKJson	Text	Nullable	Danh sách các lớp top-k ở dạng JSON.
RawResponseJson	Text	Nullable	Toàn bộ phản hồi gốc từ dịch vụ suy luận FastAPI ở dạng JSON.

Bảng 4.5. Cấu trúc bảng Feedback

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
FeedbackId	UUID	PK, Not null	Định danh duy nhất của phản hồi hậu kiểm.
PredictionId	UUID	FK -> Prediction.PredictionId, Not null	Khóa ngoại liên kết phản hồi với kết quả dự đoán tương ứng.
UserId	UUID	FK -> User.UserId, Not null	Người dùng đã gửi phản hồi cho kết quả dự đoán.
IsCorrect	Boolean	Nullable	Đánh dấu người dùng có cho rằng dự đoán là đúng hay không.
UserLabel	varchar(100)	Nullable	Nhãn do người dùng gợi ý trong trường hợp muốn điều chỉnh kết quả.
Comment	varchar(1000)	Nullable	Ghi chú hoặc nhận xét bổ sung của người dùng.
AllowForRetrain	Boolean	Not null	Xác định phản hồi có được phép dùng cho tập dữ liệu tái huấn luyện hay không.
CreatedAt	timestamp(6)	Not null	Thời điểm phản hồi được tạo trong hệ thống.

CHƯƠNG 5: TRAINING AI VÀ TRIỂN KHAI

5.1. Pipeline mô hình



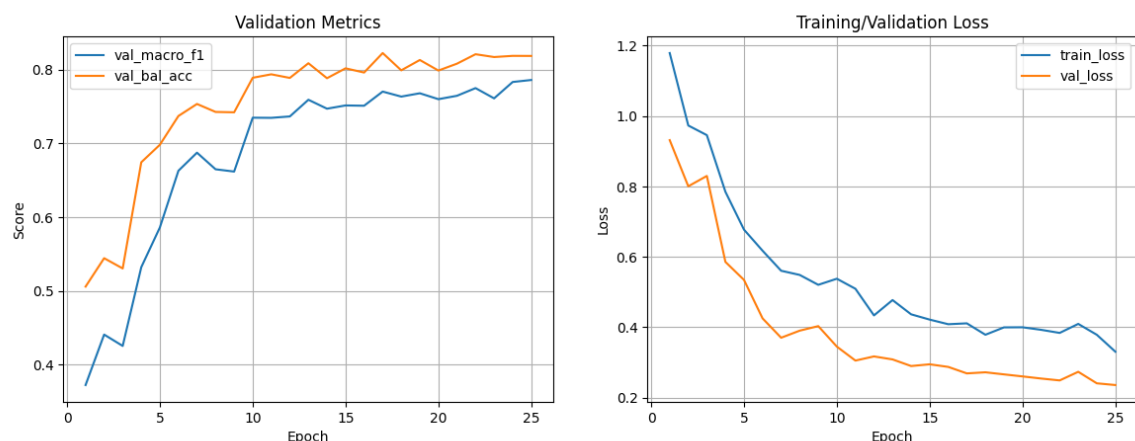
Hình 5.1. Pipeline huấn luyện và đánh giá mô hình

Bảng 5.1. Cấu hình huấn luyện

Tham số	Giá trị	Tham số	Giá trị
IMG_SIZE	300	BATCH_SIZE	32
NUM_WORKERS	2	EPOCHS	25
FREEZE_EPOCHS	3	LR_HEAD	3e-4
LR_BACKBONE	3e-5	WEIGHT_DECAY	1e-4
LABEL_SMOOTHING	0.05	USE_FOCAL	True
FOCAL_GAMMA	2.0	USE_WEIGHTED_SAMPLER	True

5.1.1. Diễn biến hội tụ trên tập validation

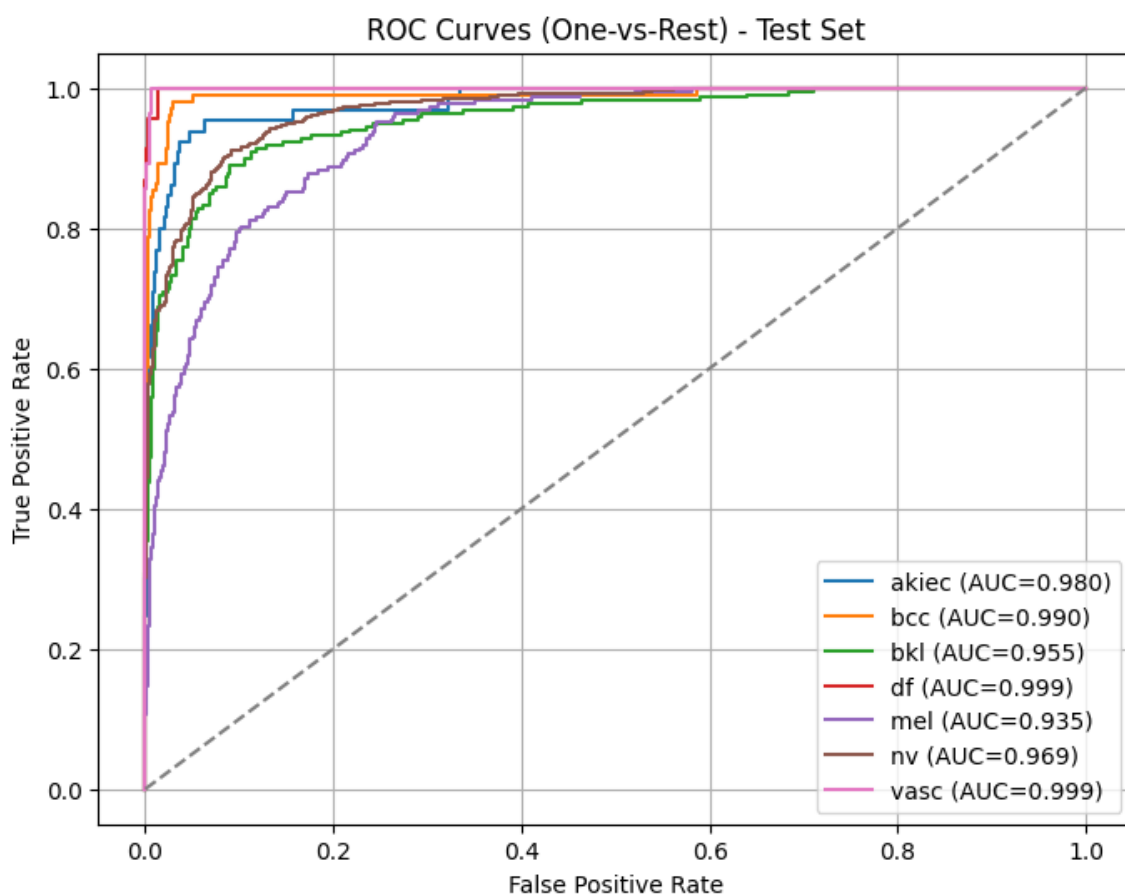
Hai đường cong validation cho thấy mô hình cải thiện rõ rệt trong các epoch đầu, sau đó ổn định dần. Macro-F1 tăng từ khoảng 0.37 lên gần 0.79, còn balanced accuracy tăng từ khoảng 0.50 lên hơn 0.82. Đồng thời, train loss và val loss đều giảm đều theo thời gian, cho thấy chiến lược regularization đang phát huy hiệu quả và chưa xuất hiện dấu hiệu overfit nghiêm trọng ở giai đoạn cuối.



Hình 5.2. Diễn biến validation metrics và training/validation loss

5.1.2. ROC curves

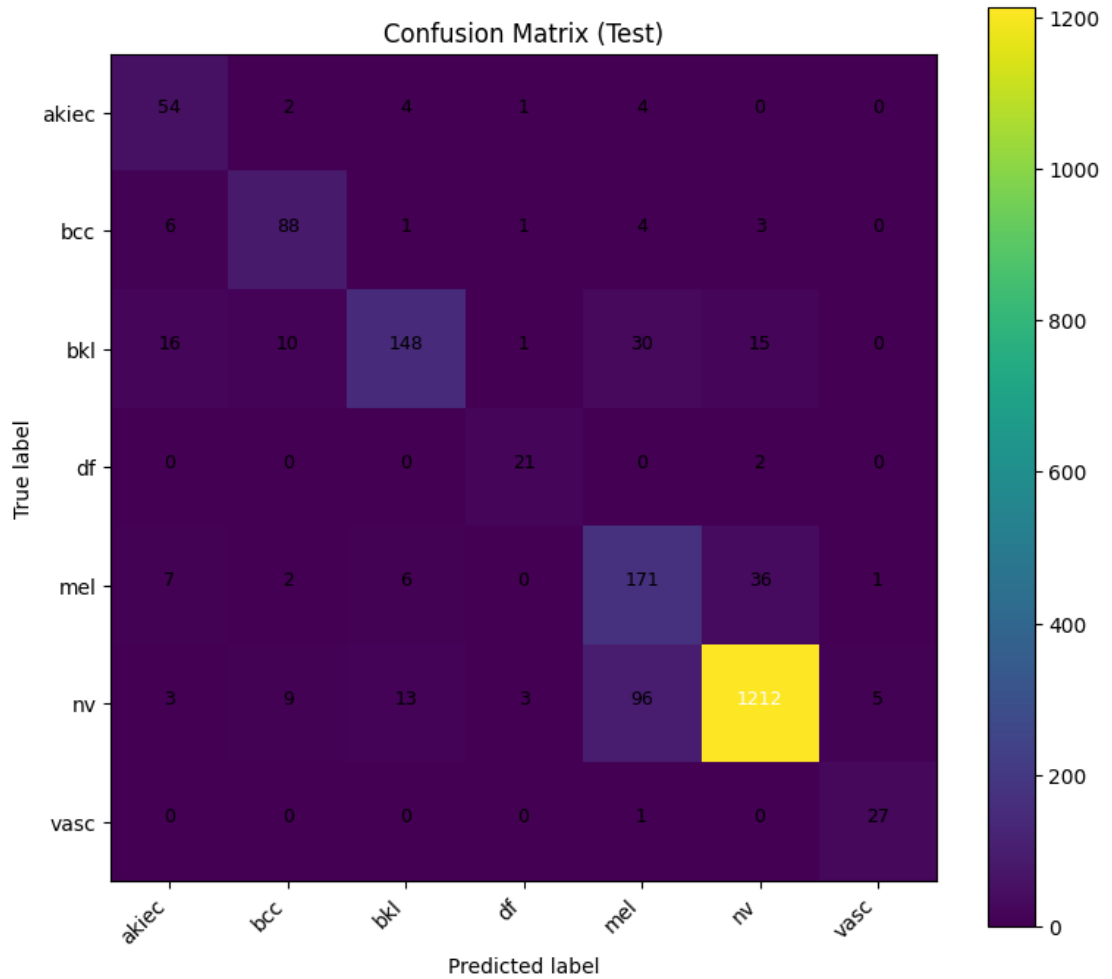
ROC one-vs-rest cho thấy hầu hết các lớp đạt AUC rất cao. Các lớp df và vasc đều xấp xỉ 0.999; bcc đạt 0.990; akiec đạt 0.980; nv đạt 0.969; bkl đạt 0.955; mel là lớp khó nhất nhưng vẫn đạt 0.935. Trung bình cộng đơn giản của 7 AUC đạt khoảng 0.9753, cho thấy khả năng tách lớp của mô hình khá tốt ngay cả khi một số lớp hiếm vẫn có độ hỗn loạn cao ở bước quyết định ngưỡng.



Hình 5.3. ROC curves (One-vs-Rest) trên test set

5.1.3. Confusion matrix

Ma trận nhầm lẫn cho phép nhìn sâu hơn vào hành vi phân lớp. Lớp nv có số lượng mẫu lớn nhất và cũng là lớp bị nhầm lẫn với mel nhiều nhất; ngược lại mel cũng bị kéo về nv với tần suất đáng kể. Lớp bkl có hiện tượng nhầm sang mel và akiec. Điều này phù hợp với thực tế rằng nhiều tổn thương da có họa tiết và sắc độ gần nhau trong ảnh dermoscopy, khiến điểm AUC cao chưa chắc đồng nghĩa với F1 tuyệt đối cao.



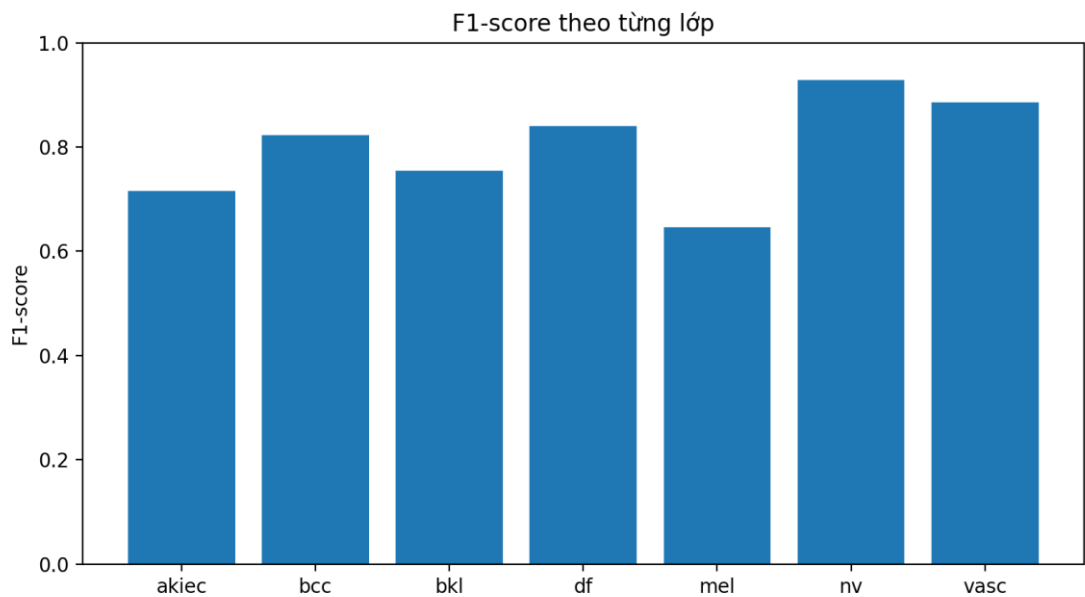
Hình 5.4. Confusion matrix trên test set

Bảng 5.2. Chỉ số đánh giá tổng hợp

Metric	Value
Accuracy	0.8592
Macro F1	0.7991
Balanced Accuracy	0.8437
Weighted F1	0.8645
Test samples	2003.0

Bảng 5.3. Độ đo theo từng lớp

Clas s	Precisi on	Recall	F1- score	Supp ort	AUC
akiec	0.6279	0.8308	0.7152	65	0.980
bcc	0.7928	0.8544	0.8224	103	0.990
bkl	0.8605	0.6727	0.7551	220	0.955
df	0.7778	0.9130	0.8400	23	0.999
mel	0.5588	0.7668	0.6465	223	0.935
nv	0.9558	0.9038	0.9291	1341	0.969
vasc	0.8182	0.9643	0.8852	28	0.999



Hình 5.5. F1-score theo từng lớp

5.2. Thảo luận kết quả

Từ các bảng trên, có thể rút ra bốn nhận xét quan trọng. Thứ nhất, mô hình đạt accuracy 0.8592 nhưng macro-F1 chỉ 0.7991, chứng tỏ accuracy không đủ đại diện trong bối cảnh mất cân bằng lớp. Thứ hai, balanced accuracy 0.8437 cao hơn macro-F1, cho thấy recall giữa các lớp tương đối tốt nhưng precision ở một số lớp khó vẫn còn hạn chế. Thứ ba, mel là lớp thách thức nhất với F1 xấp xỉ 0.6465; đây là lớp nhạy cảm về mặt y học nên cần tiếp tục cải thiện. Thứ tư, nv có F1 0.9291 và support 1341 mẫu, cho thấy mô hình học rất tốt ở lớp đa số nhưng vẫn có nguy cơ hút nhầm các lớp khó khác về nv.

- akiec: recall 0.8308 cho thấy mô hình bắt được khá nhiều mẫu thật, nhưng precision 0.6279 phản ánh còn nhầm với lớp khác.
- bcc: cân bằng tốt giữa precision 0.7928 và recall 0.8544, F1 đạt 0.8224.
- bkl: precision cao 0.8605 nhưng recall 0.6727, nghĩa là mô hình khá dè dặt khi gán nhãn bkl.
- df và vasc: recall rất cao, song support nhỏ nên chưa thể kết luận quá mạnh về tín ổn định ngoài tập test hiện tại.
- mel: AUC 0.935 vẫn tốt, nhưng F1 thấp hơn rõ rệt do sai số ở ngưỡng quyết định và nhầm lẫn với nv / bkl.

5.3. Phân tích mã huấn luyện

Mã huấn luyện mà người dùng gửi có cấu trúc rõ ràng và có thể chia thành chín khối chức năng: (1) cấu hình siêu tham số qua dataclass CFG; (2) đặt seed và thiết bị; (3) khai báo transform; (4) khởi tạo ImageFolder và DataLoader; (5) định nghĩa Focal Loss và Mixup; (6) khởi tạo EfficientNet-B3 và cơ chế freeze/unfreeze; (7) optimizer + scheduler; (8) evaluate / predict; và (9) vòng lặp huấn luyện với early stopping. Trong phụ lục, báo cáo đính kèm một bản script hoàn thiện hơn từ code bạn gửi, có bổ sung hàm vẽ ROC, confusion matrix và lưu lịch sử metric.

Bảng 5.4. Phân tích mã huấn luyện

Khối mã	Mục đích
CFG + seed_everything	Quản lý toàn bộ siêu tham số và đảm bảo tính tái lập có kiểm soát
Transforms / ImageFolder	Tiền xử lý và nạp dữ liệu theo chuẩn train/val/test
WeightedRandomSampler	Cân bằng xác suất lấy mẫu theo lớp
FocalLoss + Mixup	Giảm thiên lệch lớp và tăng khả năng tổng quát hóa
build_model + set_trainable_backbone	Thiết lập EfficientNet-B3 và hai giai đoạn fine-tune
evaluate / predict_all	Tính macro-F1, balanced accuracy và dự đoán test
Train loop	Huấn luyện, lưu checkpoint tốt nhất và early stopping
Plot utilities	Sinh ra các hình metric dùng cho báo cáo

5.4. Môi trường và cấu hình triển khai

Bảng 5.5. Môi trường và cấu hình triển khai

Thành phần	Cấu hình / công nghệ
Backend	Spring Boot 3.3.2 - Java 17 - Maven - PostgreSQL
Frontend	Static HTML/CSS/JS được phục vụ cùng backend
Inference	FastAPI + PyTorch + torchvision + uvicorn
Checkpoint	artifacts/best_model.pt
ephemeral storage	/tmp/uploads
Docker	ModelService và Backend có Dockerfile riêng

CHƯƠNG 6:

ĐÁNH GIÁ KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN

6.1. Đánh giá kết quả đạt được

Về mặt kiến trúc, dự án đạt được mức hoàn thiện cao hơn khá nhiều so với các repo học sâu thông thường vì không chỉ dừng ở bước huấn luyện mô hình, mà đã ghép được ba lớp tương đối hoàn chỉnh gồm huấn luyện mô hình, triển khai dịch vụ suy luận và backend ứng dụng. Hệ thống hiện tại được tổ chức theo hướng frontend giao tiếp với backend Spring Boot, backend tiếp tục gọi sang FastAPI ModelService để suy luận ảnh bằng checkpoint đã huấn luyện, sau đó lưu lịch sử và phản hồi vào cơ sở dữ liệu. Kiến trúc này phản ánh đúng tinh thần của một hệ thống AI ứng dụng chứ không chỉ là một mô hình chạy trong notebook hay môi trường thử nghiệm cục bộ.

Về mặt chất lượng mô hình, kết quả đạt được là tích cực nếu đặt trong bối cảnh bài toán phân loại ảnh đa liều nhiều lớp, dữ liệu mất cân bằng mạnh và có sự chồng lấn đặc trưng thị giác giữa các nhóm tổn thương. Theo kết quả thực nghiệm của đề tài, chỉ số validation macro-F1 tiến gần mức 0.80, balanced accuracy vượt 0.82, trong khi trên tập test các đường ROC one-vs-rest theo từng lớp đều đạt mức cao, trong đó nhiều lớp đạt trên 0.95. Điều này cho thấy mô hình không chỉ học được lớp phổ biến, mà còn duy trì được khả năng phân tách tốt trên phần lớn các lớp hiếm và lớp khó. Đây là kết quả đáng ghi nhận đối với một mô hình EfficientNet-B3 được huấn luyện trong phạm vi đồ án với dữ liệu HAM10000.

Về mặt sản phẩm, hệ thống cũng có nhiều điểm mạnh rõ rệt. Thứ nhất, kiến trúc được tách lớp rõ ràng theo chuỗi frontend, backend Spring Boot, FastAPI ModelService và checkpoint mô hình. Điều này giúp phần suy luận AI không bị trộn lẫn với phần nghiệp vụ ứng dụng, từ đó dễ bảo trì và dễ mở rộng hơn. Thứ hai, pipeline huấn luyện có chiều sâu kỹ thuật chứ không chỉ dừng ở transfer learning cơ bản, vì đã kết hợp WeightedRandomSampler, Focal Loss, Mixup, label smoothing, AdamW, cosine warm restarts, gradient clipping, AMP và early stopping. Thứ ba, hệ thống hỗ trợ quick-check không cần đăng nhập, giúp người dùng có thể dùng thử nhanh mà không phải đi qua toàn bộ luồng xác thực. Thứ tư, backend đã có cơ chế lưu trace khá đầy đủ như file hash, file URI, TopKJson, RawResponseJson, nhờ đó hỗ trợ tốt cho truy vết prediction và phục vụ hậu kiểm. Thứ năm, hệ thống còn có lịch sử dự đoán, feedback và retrain samples, tạo ra một vòng phản hồi quan trọng cho định hướng cải tiến mô hình trong tương lai.

Nếu xét riêng trong bối cảnh đồ án học tập, một ưu điểm rất đáng chú ý là đề tài đã giải quyết được bài toán theo hướng từ đầu đến cuối: từ chuẩn bị dữ liệu, huấn luyện mô hình, đóng gói mô hình, tổ chức API suy luận, xây dựng backend quản lý người dùng và feedback, cho tới triển khai hệ thống trên nền tảng cloud.

Đây là mức hoàn thiện cao hơn đáng kể so với các đồ án chỉ dừng ở mức báo cáo accuracy hoặc demo một giao diện đơn giản gọi mô hình.

6.2. Đánh giá so sánh với EfficientNet-B4 và EfficientNet-B5 từ nguồn công khai

Để đánh giá mô hình của đề tài một cách khách quan hơn, cần đặt lựa chọn EfficientNet-B3 vào bối cảnh so sánh với các phiên bản lớn hơn trong cùng họ mạng, cụ thể là EfficientNet-B4 và EfficientNet-B5. Về mặt lý thuyết, B4 và B5 có quy mô lớn hơn B3, đồng nghĩa với năng lực biểu diễn cao hơn. Tuy nhiên, trên dữ liệu y sinh như HAM10000, việc tăng kích thước mô hình không tự động bảo đảm chất lượng tốt hơn, đặc biệt khi dữ liệu bị mất cân bằng và số lượng ảnh trong từng lớp chênh lệch đáng kể.

Một benchmark công khai trên GitHub về HAM10000 đã huấn luyện các teacher models thuộc họ EfficientNet từ B0 đến B7 bằng pretrained weights, Focal Loss, AdamW và early stopping theo validation ROC-AUC. Trong benchmark này, EfficientNet-B3 đạt ROC-AUC 0.8883, F1 0.5614, recall 0.5749, specificity 0.9409 và accuracy 0.9003. EfficientNet-B4 đạt ROC-AUC 0.8853, F1 0.5263, recall 0.5689, specificity 0.9260 và accuracy 0.8863. EfficientNet-B5 đạt ROC-AUC 0.8834, F1 0.4892, recall 0.7425, specificity 0.8384 và accuracy 0.8278. Cùng nguồn này cũng ghi rõ quy mô mô hình và kích thước đầu vào: EfficientNet-B3 khoảng 12.0 triệu tham số với ảnh 300×300 , EfficientNet-B4 khoảng 19.3 triệu tham số với ảnh 380×380 , còn EfficientNet-B5 khoảng 30.4 triệu tham số với ảnh 456×456 .

Từ số liệu công khai trên có thể thấy một nhận xét rất quan trọng: mô hình lớn hơn không đồng nghĩa với hiệu năng tốt hơn trên HAM10000. Trong chính benchmark đó, EfficientNet-B3 không thua B4 và B5, mà còn đạt ROC-AUC cao hơn cả hai. Điều này có ý nghĩa rất lớn đối với việc đánh giá đề tài, bởi nó cho thấy lựa chọn EfficientNet-B3 là có cơ sở, không phải là phương án trung gian do thiếu tài nguyên. Nói cách khác, ngay cả khi so với các phiên bản lớn hơn trong cùng họ mạng, B3 vẫn là một điểm cân bằng tốt giữa khả năng biểu diễn và hiệu quả thực nghiệm trên nền dữ liệu này.

Ngoài ra, benchmark công khai cũng cho thấy EfficientNet-B5 có xu hướng đánh đổi specificity để lấy recall. Cụ thể, B5 có recall 0.7425 cao hơn đáng kể B3 và B4, nhưng specificity lại giảm xuống 0.8384, thấp hơn rõ rệt so với B3 là 0.9409 và B4 là 0.9260. Điều này hàm ý rằng B5 có thể nhạy hơn ở một số tình huống phát hiện lớp mục tiêu, nhưng đồng thời cũng dễ phát sinh nhiều dương tính giả hơn. Trong một đồ án học tập hướng đến sự ổn định và khả năng triển khai, đây không hẳn là một đánh đổi có lợi tuyệt đối.

Khi đặt mô hình của đề tài vào bối cảnh đó, có thể rút ra một số ưu điểm rõ rệt. Thứ nhất, việc chọn EfficientNet-B3 là hợp lý xét trên cả lý thuyết lẫn thực nghiệm công khai. B3 nhỏ hơn đáng kể so với B4 và B5, dùng input 300×300 thay vì 380×380 hoặc 456×456 , nhờ đó chi phí GPU, thời gian huấn luyện và chi phí suy luận thấp hơn nhiều, trong khi hiệu quả không bị suy giảm tương ứng. Thứ hai, đề tài không chỉ dùng B3 theo cách fine-tune đơn giản, mà còn tăng cường bằng một pipeline huấn luyện có chiều sâu gồm xử lý mất cân bằng lớp và nhiều kỹ thuật regularization, nên giá trị học thuật của mô hình trong đồ án còn cao hơn kiểu “lấy pretrained model rồi train nhanh vài epoch”. Thứ ba, do hệ thống của đề tài đã được triển khai thành backend + inference service, lựa chọn B3 còn có ý nghĩa ở mặt engineering: mô hình đủ mạnh để cho kết quả tốt nhưng vẫn khả thi khi phục vụ suy luận online qua API.

Bên cạnh đó, mô hình của đề tài vẫn còn một số điểm hạn chế nếu nhìn từ góc độ so sánh học thuật. Thứ nhất, đồ án chưa có thí nghiệm nội bộ đối chứng trực tiếp giữa B3, B4 và B5 trên đúng cùng tập chia dữ liệu, cùng pipeline augmentation, cùng sampler và cùng tiêu chí chọn checkpoint. Vì vậy, dù benchmark công khai ủng hộ B3, đồ án vẫn chưa chứng minh bằng thực nghiệm riêng rằng B4 hoặc B5 chắc chắn không cải thiện được một số lớp khó trong chính pipeline hiện tại. Thứ hai, trong kết quả của đề tài, lớp mel vẫn là lớp khó và còn bị nhầm lẫn đáng kể với nv. Đây là tình huống mà về mặt lý thuyết, các mô hình lớn hơn như B4 hoặc B5 có thể vẫn đáng để thử trong các nghiên cứu tiếp theo, đặc biệt nếu có thêm tài nguyên và chiến lược regularization phù hợp. Thứ ba, benchmark công khai được dùng để tham chiếu là bài toán trên cùng nền dữ liệu HAM10000 nhưng khác thiết lập mục tiêu đánh giá, nên phần so sánh này cần được hiểu là so sánh học thuật và tham khảo định hướng, không phải đối chứng tuyệt đối.

Tóm lại, trong ngữ cảnh của một đồ án học tập, lựa chọn EfficientNet-B3 của đề tài có thể được xem là hợp lý và có cơ sở. So với B4 và B5 trong benchmark công khai, B3 đạt hiệu năng không hề thua rõ rệt, trong khi nhẹ hơn đáng kể và phù hợp hơn với mục tiêu triển khai thành hệ thống hoàn chỉnh. Đây là một ưu thế lớn của đề tài: không chạy theo mô hình lớn nhất, mà lựa chọn mô hình cân bằng nhất giữa chất lượng, tài nguyên và khả năng tích hợp thực tế.

6.3. Những mặt còn hạn chế

Bên cạnh các điểm mạnh, hệ thống vẫn mang một số dấu ấn của giai đoạn nguyên mẫu và triển khai học thuật. Một số cấu hình trong project ban đầu vẫn có xu hướng phụ thuộc môi trường local, chẳng hạn đường dẫn lưu trữ file hoặc giá trị mặc định của một số biến cấu hình. Dù hệ thống hiện đã được triển khai trên Render, việc chuẩn hóa hoàn toàn các biến cấu hình giữa môi trường local và môi trường cloud vẫn là một yêu cầu cần tiếp tục hoàn thiện để giảm sai khác giữa phát triển và vận hành thực tế.

Ở tầng mô hình, lớp mel vẫn là lớp khó nhất trong các nhóm tổn thương da. Confusion matrix cho thấy còn sự nhầm lẫn đáng kể giữa mel và nv, đồng thời một phần bkl cũng dễ bị nhầm sang các lớp có sắc tố gần tương tự. Điều này cho thấy dù mô hình đã đạt kết quả tốt trên phần lớn các lớp, bài toán cốt lõi của ảnh da liễu vẫn chưa được giải quyết triệt để. Đây là hạn chế có tính bản chất, không chỉ của riêng đề tài mà còn là khó khăn chung của nhiều nghiên cứu dùng HAM10000.

Ở tầng backend, cách chuẩn hóa phản hồi từ FastAPI theo hướng thực dụng giúp hệ thống dễ chạy ổn định hơn, nhưng cũng có khả năng che khuất các lỗi contract nếu hai phía backend và model service bị lệch cấu trúc dữ liệu trong tương lai. Tức là hệ thống hiện ưu tiên vận hành được và xử lý linh hoạt, nhưng nếu mở rộng quy mô lớn hơn thì cần bổ sung kiểm tra contract chặt chẽ hơn giữa các service.

Một hạn chế khác là hiện chưa có lớp kiểm thử tự động đầy đủ cho toàn bộ stack. Hệ thống đã có cấu trúc khá tốt để phát triển tiếp, nhưng chưa thấy một pipeline CI/CD hoàn chỉnh bao gồm unit test, integration test và contract test giữa backend với ModelService. Trong bối cảnh một đồ án, điều này vẫn chấp nhận được, nhưng nếu xét theo chuẩn hệ thống phần mềm sản xuất thì đây là phần còn thiếu.

Ngoài ra, cơ chế loại ảnh không phải da hiện nghiêng về heuristic và quy tắc thực dụng. Cách làm này hữu ích trong giai đoạn đầu để giảm ảnh rác đi vào mô hình, nhưng về lâu dài có thể bị ảnh hưởng bởi sự đa dạng của ánh sáng, tông màu da, thiết bị chụp hoặc bối cảnh ảnh. Nghĩa là tính tổng quát của tầng tiền kiểm tra ảnh hiện chưa mạnh bằng tầng mô hình chính.

Phần đánh giá thực nghiệm cũng chưa đi sâu vào các nội dung như calibration, uncertainty estimation, phân tích độ tin cậy dự đoán trên dữ liệu ngoài miền hoặc đánh giá độ ổn định của xác suất. Trong các hệ thống AI ứng dụng vào y tế, đây là những yếu tố rất quan trọng nếu muốn tiến thêm từ mức đồ án sang mức công cụ hỗ trợ có độ tin cậy cao hơn.

6.4. Hướng phát triển trong tương lai

Từ cấu trúc hiện tại, hệ thống có nhiều hướng phát triển thực tế và có thể mở rộng tốt cả ở mức kỹ thuật lẫn mức sản phẩm. Ở tầng triển khai, có thể chuẩn hóa toàn bộ cấu hình theo biến môi trường hoặc .env, đồng thời đồng bộ nhất quán giữa local, Docker và cloud deployment. Nếu hệ thống cần mở rộng thêm, có thể tiến tới các mô hình orchestration cao hơn thay vì chỉ dừng ở mức Docker hoặc Render service tách rời.

Ở tầng backend, một hướng phát triển quan trọng là bổ sung test unit và integration test cho các service chính, đồng thời viết contract test giữa backend Spring Boot và FastAPI ModelService. Việc này sẽ giúp phát hiện sớm các sai lệch

Ở tầng quản trị, hệ thống có thể được mở rộng thêm dashboard admin chuyên sâu hơn, chẳng hạn phân tích confusion pattern, tần suất feedback theo từng lớp, tỷ lệ feedback đúng/sai theo thời gian, hay dấu hiệu drift trong dữ liệu người dùng thật. Đây là bước phát triển phù hợp vì hệ thống của đề tài đã có sẵn dữ liệu lịch sử prediction và feedback, tức là đã có nền để xây dựng các phân tích này.

Ở tầng mô hình, có thể nghiên cứu thêm các kỹ thuật như segmentation, attention mechanism hoặc các mô hình calibration để làm cho xác suất dự đoán đáng tin cậy hơn. Một hướng có giá trị thực tế cao là áp dụng temperature scaling hoặc các kỹ thuật calibration tương tự, vì trong y tế, độ tin cậy của xác suất quan trọng không kém nhãn phân loại cuối cùng.

Ngoài ra, dữ liệu feedback hiện tại nên được phát triển thành một luồng gắn nhãn bán tự động có kiểm soát. Nếu làm tốt, đây sẽ là nền tảng để hệ thống hình thành một vòng phản hồi khép kín: dự đoán – lưu vết – nhận phản hồi – chọn mẫu retrain – huấn luyện lại mô hình. Đây là hướng phát triển rất phù hợp với bản chất của một hệ thống AI ứng dụng lâu dài.

Sau cùng, hệ thống cần tiếp tục tăng cường thông điệp an toàn y tế. Dù về kỹ thuật mô hình có thể tiếp tục được cải thiện, hệ thống vẫn nên luôn được định vị là công cụ hỗ trợ tham khảo, không thay thế bác sĩ chuyên khoa. Việc nhấn mạnh rõ ranh giới này không chỉ đúng về mặt đạo đức ứng dụng AI mà còn làm cho sản phẩm của đề tài có định hướng sử dụng thực tế hơn.

6.5. Kết luận chương

Nhìn chung, kết quả đạt được của đề tài là tích cực trên cả ba phương diện: mô hình, hệ thống và khả năng triển khai. Ở phương diện mô hình, EfficientNet-B3 kết hợp với pipeline huấn luyện phù hợp đã cho kết quả tốt trên bài toán phân loại đa lớp của HAM10000. Ở phương diện hệ thống, project đã vượt ra khỏi phạm vi của một mô hình học sâu đơn lẻ để trở thành một nền tảng có backend, inference service, lịch sử prediction, feedback và cơ chế chuẩn bị dữ liệu retrain. Ở phương diện triển khai, hệ thống đã được đưa lên Render, cho thấy tính khả thi trong vận hành thực tế.

Khi so sánh với các benchmark công khai dùng EfficientNet-B4 và EfficientNet-B5 trên HAM10000, có thể thấy lựa chọn EfficientNet-B3 của đề tài là hợp lý trong bối cảnh đồ án học tập: mô hình đủ mạnh để cho kết quả tốt, nhẹ hơn đáng kể so với các phiên bản lớn hơn, và thuận lợi hơn cho triển khai thành hệ thống hoàn chỉnh. Tuy nhiên, bài toán vẫn còn các hạn chế chưa được giải quyết triệt để, đặc biệt ở lớp khó như mel, ở tính tổng quát của tầng kiểm tra ảnh đầu vào và ở chiều sâu đánh giá độ tin cậy của mô hình. Những hạn chế này đồng thời cũng mở ra các hướng phát triển rõ ràng cho các giai đoạn nghiên cứu tiếp theo.

TÀI LIỆU THAM KHẢO

- [1] Philipp Tschandl, Cliff Rosendahl, Harald Kittler. *The HAM10000 Dataset: A Large Collection of Multi-Source Dermatoscopic Images of Common Pigmented Skin Lesions*. Scientific Data, 2018.
Link: <https://www.nature.com/articles/sdata2018161>
- [2] International Skin Imaging Collaboration (ISIC). *ISIC Archive*.
Link: <https://www.isic-archive.com/>
- [3] Mingxing Tan, Quoc V. Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. Proceedings of the 36th International Conference on Machine Learning, 2019. Link: <https://proceedings.mlr.press/v97/tan19a.html>
- [4] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, Piotr Dollár. *Focal Loss for Dense Object Detection*. IEEE Transactions on Pattern Analysis and Machine Intelligence / ICCV 2017 version.
Link: <https://arxiv.org/abs/1708.02002>
- [5] Hongyi Zhang, Moustapha Cissé, Yann N. Dauphin, David Lopez-Paz. *mixup: Beyond Empirical Risk Minimization*. International Conference on Learning Representations, 2018. Link: <https://arxiv.org/abs/1710.09412>
- [6] Ilya Loshchilov, Frank Hutter. *Decoupled Weight Decay Regularization*. International Conference on Learning Representations, 2019. Link: <https://arxiv.org/abs/1711.05101>
- [7] Ryan Healy và cộng sự. *HAM10000 Melanoma Detection - Model Benchmarks*. GitHub repository notebook benchmark, so sánh các mô hình EfficientNet B0–B7 trên HAM10000
Link: <https://github.com/rah-ds/Melanoma-Detection-with-Knowledge-.ipynb>